

BDS701: Parallel Programming

Q1: Explain Flynn's taxonomy of parallel computers. Compare SIMD and MIMD systems with suitable examples.

ANS:

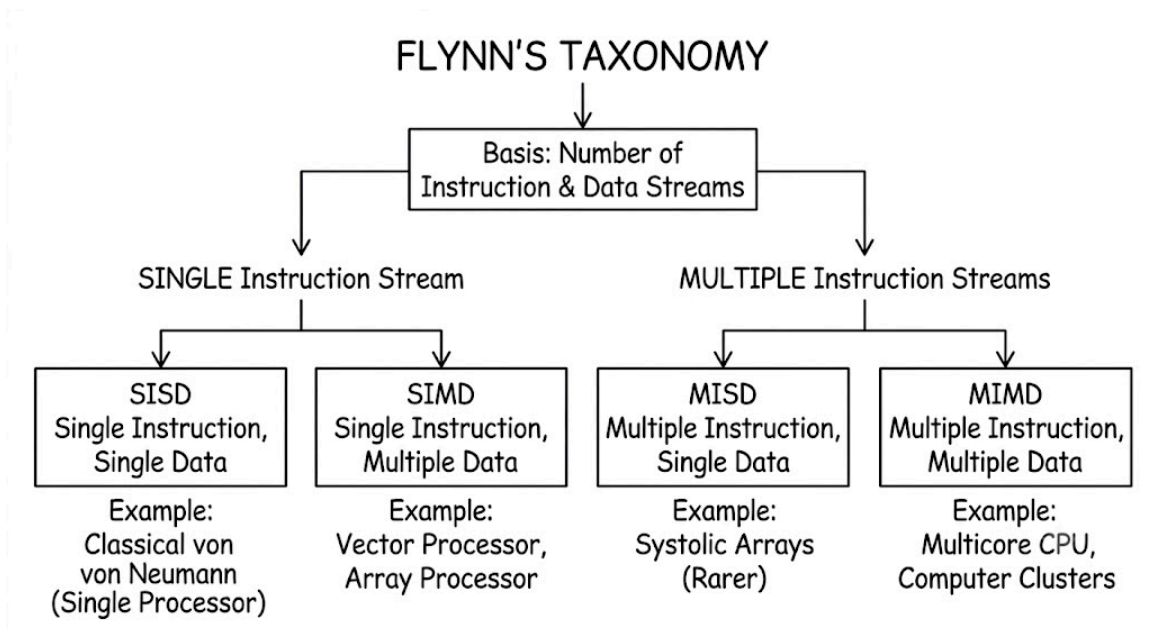
Introduction

Flynn's taxonomy is a classification of parallel computers based on the **number of instruction streams and data streams (or datapaths)** that a computer can simultaneously manage.

A classical von Neumann system is classified as **SISD (Single Instruction, Single Data)**. Parallel systems can manage multiple data streams and are differentiated into systems supporting a single instruction stream (**SIMD**) and systems supporting multiple instruction streams (**MIMD**).

Flynn's Taxonomy

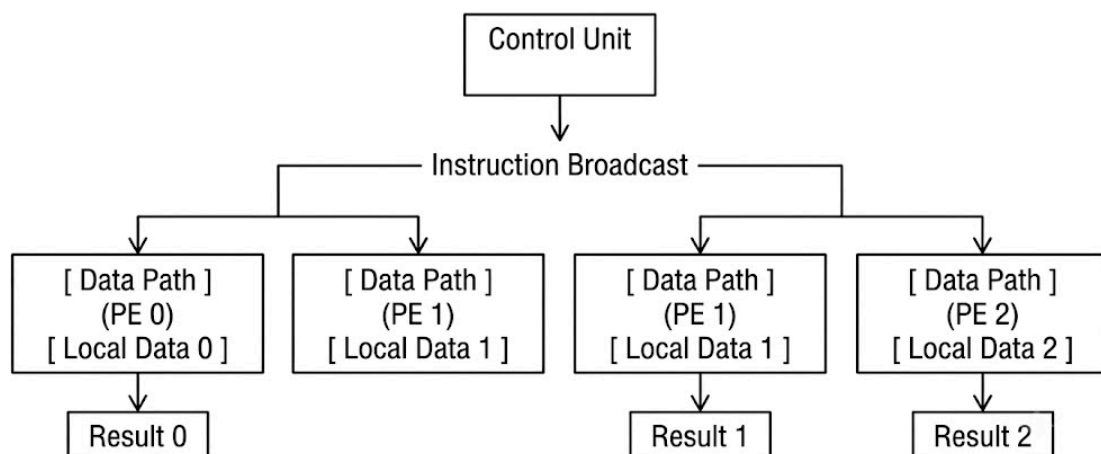
The classification can be represented as follows:



SIMD Systems

SIMD (Single Instruction, Multiple Data) systems operate on multiple data streams by applying the **same instruction to multiple data items**. An abstract SIMD system can therefore be viewed as having a **single control unit and multiple datapaths**.

SIMD: Single Instruction, Multiple Data - Expanded Block View



This diagram illustrates the parallel processing nature of a SIMD architecture, where a single Control Unit broadcasts instructions to multiple processing elements, each operating on its own unique data set (Local Data 0, 1, 2) to produce distinct results (Result 0, 1, 2) simultaneously.

The instruction is broadcast from the control unit to the datapaths. Each datapath applies the instruction to its current data item, or remains idle.

Example: Vector Addition

Consider two arrays `x` and `y`, each containing `n` elements:

```
for (i = 0; i < n; i++)  
    x[i] += y[i];
```

If the SIMD system has `n` datapaths, `x[i]` and `y[i]` can be loaded into the corresponding datapath, the addition can be performed, and the result stored in `x[i]`.

If there are fewer datapaths than data elements, the operations are performed in blocks. For example, with **4 datapaths and 15 elements**, the elements are processed as:

```
Elements 0-3  
Elements 4-7  
Elements 8-11  
Elements 12-14
```

In the final group, only three datapaths are required, so one datapath remains idle.

Characteristics of SIMD

1. A **single instruction stream** operates on multiple data streams.
2. It has a **single control unit and multiple datapaths**.
3. Instructions are broadcast to the datapaths.
4. Classical SIMD datapaths operate **synchronously**.
5. Datapaths do not have instruction storage.
6. SIMD is particularly suitable for **simple loops operating on large arrays of data**.
7. Parallelism obtained by dividing data among processors and applying approximately the same instructions to their subsets is called **data-**

parallelism.

Limitation of SIMD

If different data items require different execution paths, some datapaths may remain idle. For example:

```
if (y[i] > 0.0)
    x[i] += y[i];
```

When `y[i]` is not positive, the corresponding datapath remains idle while others perform the addition. This can seriously reduce SIMD performance.

Examples of SIMD

The **vector processors** are an important form of SIMD computing. They operate on arrays or vectors of data rather than individual scalar elements.

The **GPUs and desktop CPUs make use of aspects of SIMD computing.**

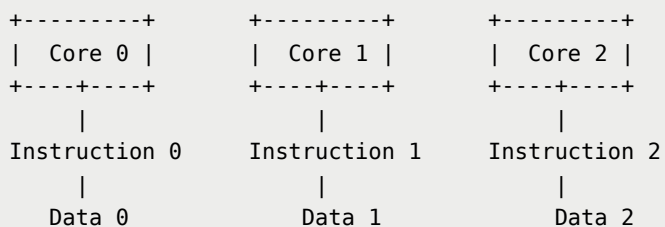
MIMD Systems

MIMD (Multiple Instruction, Multiple Data) systems support **multiple simultaneous instruction streams operating on multiple data streams.**

MIMD systems typically consist of a collection of **independent processing units or cores**, with each core having its own:

- Control unit
- Datapath

Working of MIMD



Unlike SIMD systems, MIMD processors are usually **asynchronous** and can operate at their own pace. There may be no global clock relationship between different processors. Unless synchronization is imposed, processors executing the same sequence can be executing different statements at a given instant.

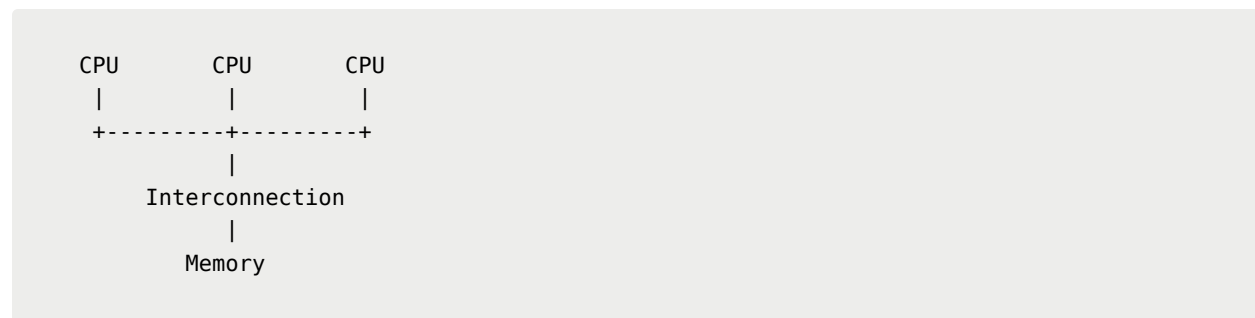
Types of MIMD Systems

There are two principal types:

a) Shared-memory systems

A collection of autonomous processors is connected to a memory system through an interconnection network, and each processor can access each memory location.

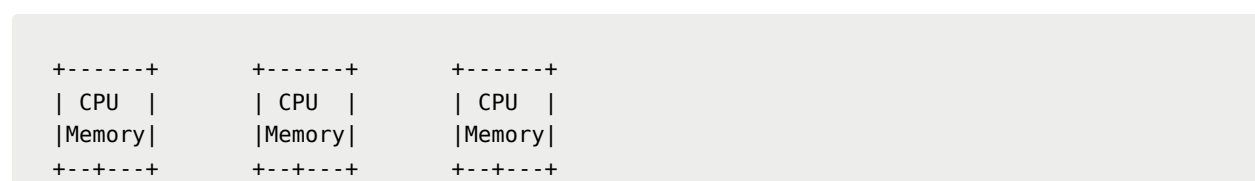
Processors generally communicate **implicitly by accessing shared data structures**.

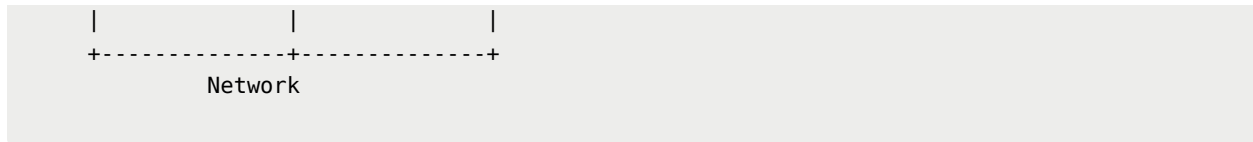


Example: shared-memory systems using **multicore processors**, where multiple CPUs or cores are present on a single chip.

b) Distributed-memory systems

Each processor is paired with its own **private memory**, and processor-memory pairs communicate through an interconnection network. Communication is generally explicit through messages or special functions providing access to another processor's memory.





Example: distributed-memory **clusters**, composed of commodity systems such as PCs connected by a commodity interconnection network such as Ethernet.

5. Comparison Between SIMD and MIMD

Feature	SIMD	MIMD
Full form	Single Instruction, Multiple Data	Multiple Instruction, Multiple Data
Instruction streams	Single	Multiple
Data streams	Multiple	Multiple
Basic organization	Single control unit with multiple datapaths	Independent processing units/cores
Instruction execution	Same instruction applied to multiple data items	Different instruction streams can operate simultaneously
Execution	Classical SIMD operates synchronously	Usually asynchronous
Control	Instruction is broadcast from control unit	Each processing unit has its own control unit
Suitability	Simple loops operating on large arrays	Problems involving multiple independent instruction streams
Parallelism	Mainly data-parallelism	Supports multiple independent instruction streams and data streams
Example	Vector processors; GPUs and desktop CPUs use aspects of SIMD	Multicore shared-memory systems and distributed-memory clusters

The distinction between SIMD and MIMD follows directly from the number of instruction streams and data streams managed by the system.

Question 2

Discuss shared-memory and distributed-memory architectures with neat diagrams. Compare their advantages, disadvantages and scalability.

Answer

1. Introduction

In a **shared-memory system**, the cores can share access to memory locations and coordinate their work by modifying shared memory locations.

In a **distributed-memory system**, each core has its own **private memory**, and the cores coordinate their work by communicating across a network.

These are the two principal types of MIMD systems.

2. Shared-Memory Architecture

A shared-memory system consists of a collection of autonomous processors connected to a **common memory system through an interconnection network**. Each processor can access each memory location.

The processors generally communicate **implicitly** by accessing shared data structures.

Diagram

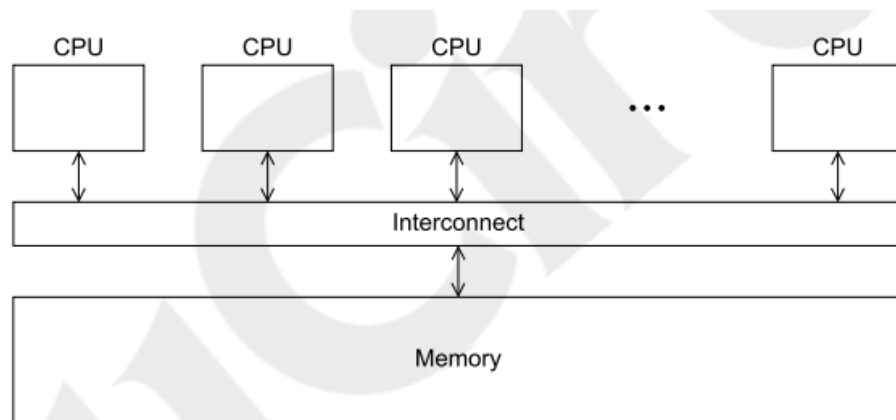


FIGURE 2.3

A shared-memory system.

Explanation: All CPUs are connected through an interconnection network to the same memory system. Therefore, a processor can access any memory location.

Characteristics

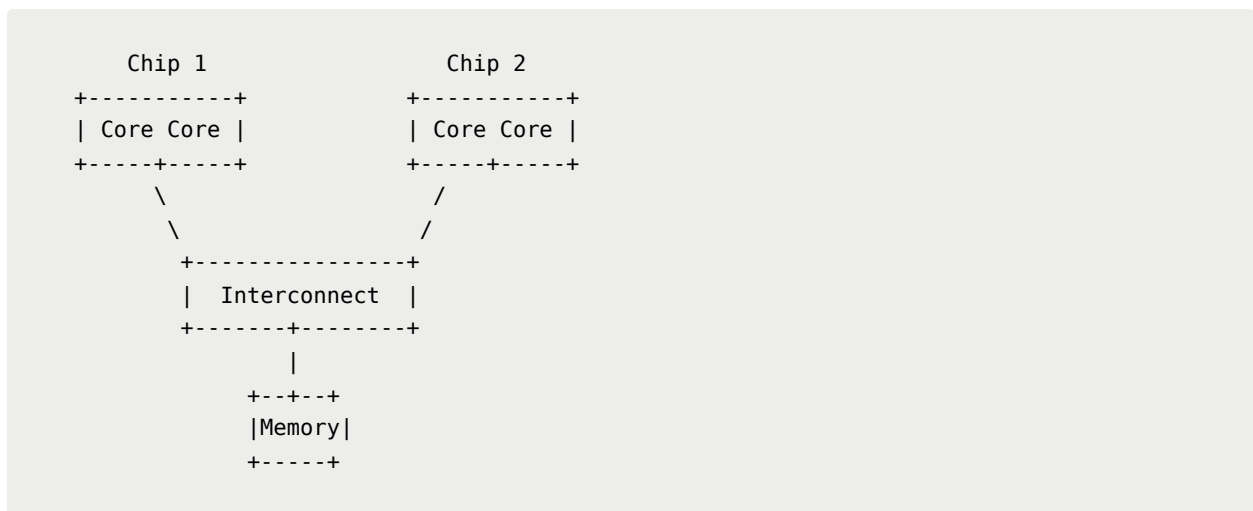
1. Multiple processors/cores share a common memory system.
2. Each processor can access each memory location.
3. Processors communicate implicitly through shared data structures.
4. Shared-memory systems widely use **multicore processors**.
5. A multicore processor contains multiple CPUs or cores on a single chip.
6. Cores typically have private level-1 caches, while other caches may or may not be shared.

3. Types of Shared-Memory Systems

In shared-memory systems with multiple multicore processors, the interconnect can be organized in two ways.

a) Uniform Memory Access : UMA

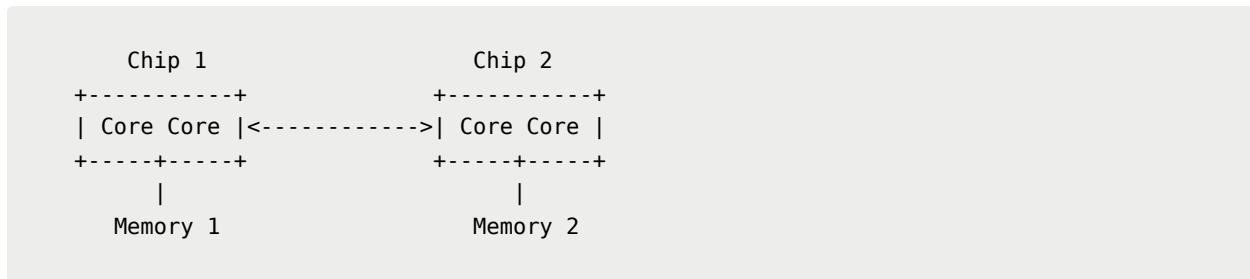
In a **UMA system**, all processors have the same access time to memory locations.



In this arrangement, the time to access all memory locations is the same for all cores.

b) Nonuniform Memory Access : NUMA

In a **NUMA system**, each processor has a direct connection to a block of main memory. A processor can also access another processor's memory through special hardware.



Memory directly connected to a core can be accessed more quickly than memory that must be accessed through another chip.

4. Advantages of Shared-Memory Architecture

1. **Easy programming:** Shared-memory systems are generally easier to program because processors can coordinate through shared data structures.
2. **Implicit communication:** Processors communicate by accessing shared data structures rather than explicitly sending messages.
3. **UMA simplicity:** UMA systems are easier to program because the programmer does not need to consider different access times for different memory locations.
4. **Multicore support:** Shared-memory systems are widely available using one or more multicore processors.

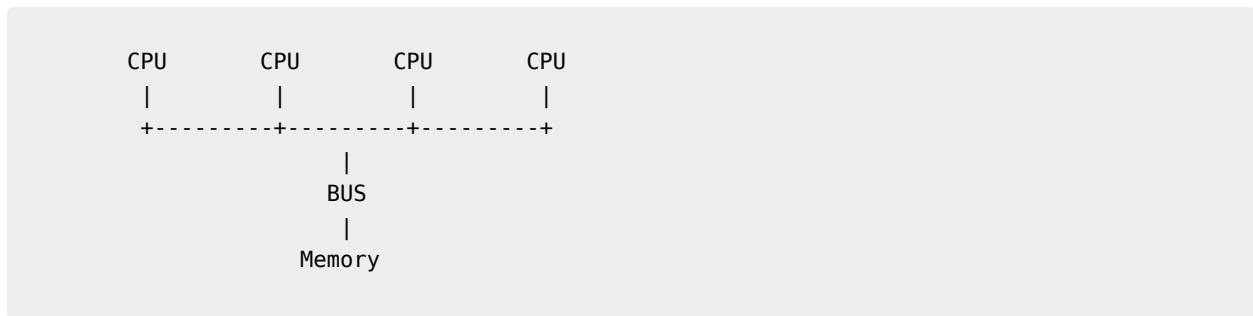
5. Disadvantages and Scalability of Shared Memory

The major issue with shared-memory systems is the **cost of scaling the interconnect**.

Bus-based scalability problem

When processors are connected using a bus, the communication wires are shared. As the number of devices increases, contention for the bus increases and

performance decreases. Processors may therefore have to wait for access to main memory.



For this reason, buses are suitable mainly for systems with a small number of processors.

Crossbar scalability problem

Switched interconnects such as a **crossbar** can provide simultaneous communication and are much faster than buses, but the switches and links are relatively expensive.

Large crossbars are therefore expensive and are uncommon for very large systems.

NUMA scalability

NUMA can provide faster access to directly connected memory and has the potential to use larger amounts of memory than UMA. However, access to memory connected to another chip is slower.

Thus, although shared memory is convenient to program, **scaling to very large numbers of processors is difficult because of interconnect cost and contention.**

6. Distributed-Memory Architecture

In a distributed-memory system, **each processor is paired with its own private memory**. The processor-memory pairs communicate through an **interconnection network**.

Communication is generally **explicit**, using messages or special functions that provide access to the memory of another processor.

Diagram

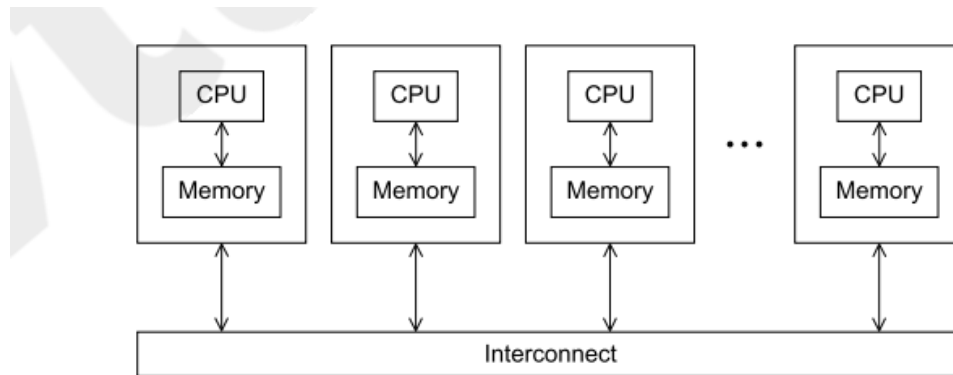


FIGURE 2.4

A distributed-memory system.

Explanation: Each processor has its own private memory. The processors communicate through the interconnection network rather than accessing one common memory.

7. Distributed-Memory Systems : Examples

The most widely available distributed-memory systems are called **clusters**.

A cluster consists of a collection of commodity systems, such as PCs, connected by a commodity interconnection network, such as **Ethernet**. The nodes are usually themselves shared-memory systems containing one or more multicore processors.

Such systems are sometimes called **hybrid systems**, because their nodes contain shared-memory systems.

Another example is a **grid**, which provides infrastructure for turning large networks of geographically distributed computers into a unified distributed-memory system. Such systems are generally heterogeneous, with nodes built from different types of hardware.

8. Advantages of Distributed-Memory Architecture

1. **Better scalability:** Distributed-memory systems can be constructed using relatively inexpensive interconnects such as the hypercube and toroidal mesh.

2. **Large systems:** Distributed-memory systems with **thousands of processors** have been built.
 3. **Large amounts of data and computation:** They are often better suited for problems requiring vast amounts of data or computation.
 4. **Private memory:** Each processor has its own private memory, reducing the requirement for all processors to access a single common memory system.
-

9. Disadvantages of Distributed-Memory Architecture

1. **Explicit communication:** Processors generally need to communicate explicitly by sending messages or using special functions to access another processor's memory.
 2. **Interconnection dependency:** Performance depends strongly on the interconnection network. A slow interconnect can seriously degrade overall parallel-program performance.
 3. **Communication overhead:** Communication between processor-memory pairs takes place through the interconnection network rather than through a common memory system.
 4. **Programming complexity:** Compared with shared memory, coordination through explicit communication is less straightforward. The notes specifically contrast the appeal of implicit shared-memory coordination with explicit message communication.
-

10. Comparison of Shared-Memory and Distributed-Memory Architectures

Feature	Shared-Memory	Distributed-Memory
Memory organization	Common/shared memory	Each processor has private memory
Processor access	Each processor can access each memory location	Processor primarily uses its own memory

Feature	Shared-Memory	Distributed-Memory
Communication	Implicit through shared data structures	Explicit through messages/special functions
Interconnection	Processors connected to common memory through interconnect	Processor-memory pairs connected through interconnect
Programming	Generally easier	Requires explicit communication
Scalability	More difficult to scale to very large systems	Better suited for large systems
Interconnect issue	Bus contention and expensive large crossbars limit scaling	Interconnects such as hypercube and toroidal mesh can be relatively inexpensive
System size	More suitable for smaller systems	Systems with thousands of processors have been built
Examples	Multicore shared-memory systems	Clusters and grids
Large computation/data	Less suited when very large scale is required	Often better suited for vast amounts of data or computation

The principal hardware distinction in scalability is the **cost of scaling the interconnect**. Shared-memory systems face bus contention and expensive large crossbars, whereas distributed-memory interconnects such as hypercubes and toroidal meshes are relatively inexpensive and have been used in systems containing thousands of processors.

Question 3

Explain cache coherence in shared-memory systems. Discuss snooping and directory-based cache-coherence mechanisms with suitable diagrams.

Answer

1. Introduction

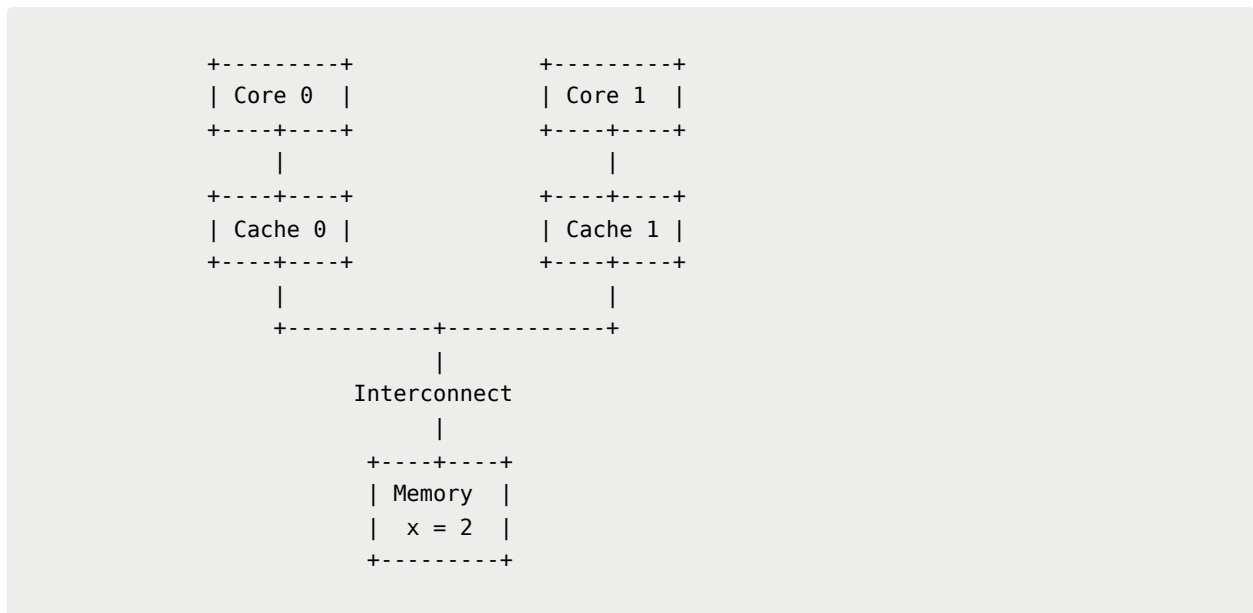
In a shared-memory system, multiple cores may have their own **private data caches** while accessing shared memory. When multiple caches contain copies of the same shared variable, an update made by one core must be properly reflected to the other cores.

The problem occurs because CPU caches are managed by system hardware and the programmer does not have direct control over when cache contents are updated.

2. Cache Coherence Problem

Consider a shared-memory system containing **two cores**, each with its own private data cache.

Diagram — Shared-memory system with two caches



Example of the problem

Suppose:

- `x` is a shared variable initialized to `2`.
- `y0` is private to Core 0.
- `y1` and `z1` are private to Core 1.

The following operations are performed:

Time	Core 0	Core 1
0	<code>y0 = x;</code>	<code>y1 = 3*x;</code>
1	<code>x = 7;</code>	Statement(s) not involving x
2	Statement(s) not involving x	<code>z1 = 4*x;</code>

Initially, both cores may have their own cached copy of `x`.

The expected values are:

```
y0 = 2  
y1 = 6
```

However, the value obtained for `z1` is not necessarily `28`. Since `x` was already present in Core 1's cache with value `2`, Core 1 may continue using that cached value and obtain:

```
z1 = 4 * 2 = 8
```

even though Core 0 has updated `x` to `7`.

This unpredictable behavior can occur with both **write-through and write-back** cache policies. With write-through, main memory is updated, but Core 1's cached copy may remain unchanged. With write-back, the new value in Core 0's cache may not yet be available to Core 1.

Definition

The **cache coherence problem** occurs when multiple processors have cached copies of the same variable and an update by one processor is not properly seen by the other processors. In other words, the cached copy stored by another processor is not updated appropriately.

3. Cache-Coherence Mechanisms

There are **two main approaches** for ensuring cache coherence:

1. **Snooping cache coherence**
 2. **Directory-based cache coherence**
-

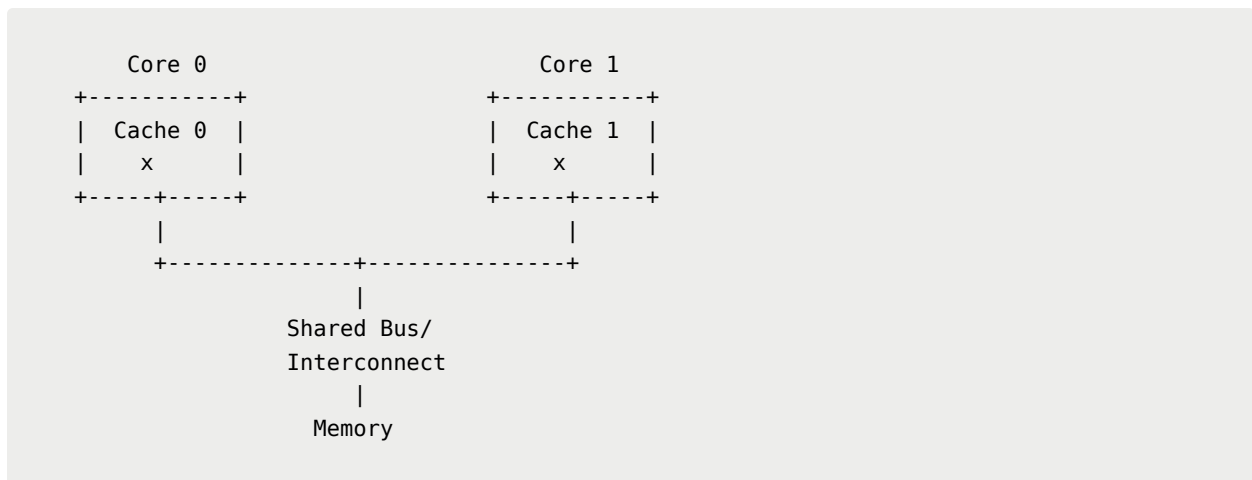
4. Snooping Cache Coherence

Definition

Snooping cache coherence is based on the idea that when cores share an interconnect, signals transmitted over the interconnect can be **seen by all the connected cores**.

Working

Consider two cores containing copies of the same shared variable x.



The working is as follows:

Step 1: Core 0 updates its cached copy of x.

Step 2: Core 0 broadcasts information about the update across the interconnect.

Step 3: Core 1, which is **snooping** the interconnect, observes the update.

Step 4: Core 1 can mark its cached copy of the corresponding cache line as **invalid**.

Thus, stale copies of the data can be prevented from being used.

Important: The actual snooping protocol broadcasts information that the **cache line containing  has been updated**, rather than simply broadcasting that the individual variable  has changed.

Important points

- Snooping is based on **broadcast communication**.
- All cores connected to the interconnect can observe the broadcast.
- A core detecting that its copy has become stale can invalidate its cache line.
- The interconnect does not necessarily have to be a bus; it must support broadcasts from processors to the other processors.

Write-through and Write-back

Snooping works with both **write-through and write-back caches**.

- With a shared interconnect and write-through caches, additional traffic may not be required because each core can watch for writes.
- With write-back caches, additional communication is required because cache updates are not immediately sent to memory.

5. Limitation of Snooping

The major limitation of snooping is **scalability**.

In large networks, broadcasts are expensive. Snooping requires a broadcast whenever a variable is updated. Therefore, as the system becomes larger, performance can degrade.

Small system:

```
Core 0 ----+
Core 1 ----+----- Shared Interconnect
Core 2 ----+
           ↓
        Broadcast manageable
```

Large system:

```
Core 0 ----+
Core 1 ----+
Core 2 ----+
Core 3 ----+
...      +---- Large Broadcast Traffic
Core N ----+
          ↓
        Performance degradation
```

Therefore:

Snooping cache coherence is not scalable for very large systems because broadcasts across the interconnect become expensive.

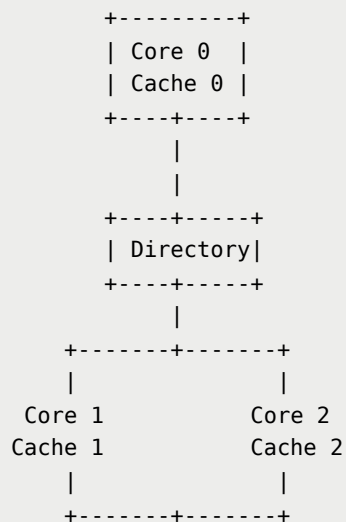
6. Directory-Based Cache Coherence

Definition

Directory-based cache coherence uses a data structure called a **directory** to maintain the coherence information.

The directory stores the **status of each cache line**.

Diagram



|
Memory

The directory keeps information about which cores have copies of particular cache lines.

7. Working of Directory-Based Coherence

Step 1: Reading a cache line

Suppose a cache line is read into Core 0's cache.

The corresponding entry in the directory is updated to indicate that **Core 0 has a copy of that cache line.**

Before:

Directory
Cache line X → No recorded copy

After Core 0 reads X:

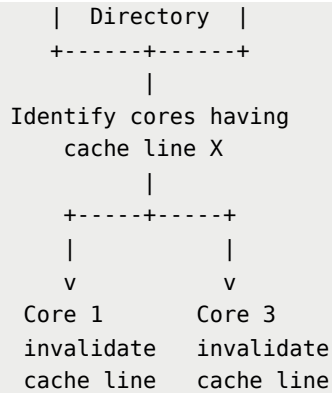
Directory
Cache line X → Core 0 has a copy

Step 2: Updating a variable

When a variable is updated:

1. The directory is consulted.
2. The directory identifies the cores that have the cache line containing that variable.
3. The cache controllers of those cores are contacted.
4. Those cache lines are invalidated.

Update X
|
v
+-----+



Therefore, instead of broadcasting the update to every core, the directory identifies the cores that actually possess the relevant cache line.

8. Distributed Directory

The directory is typically **distributed**.

Each core/memory pair may be responsible for storing the portion of the directory that specifies the status of cache lines in its local memory.

This allows the system to keep track of where copies of cache lines exist.

9. Advantages of Directory-Based Cache Coherence

1. **Avoids global broadcasts:** It does not require an update to be broadcast to every core.
 2. **Better scalability:** It addresses the scalability problem of snooping in large networks.
 3. **Selective communication:** When a cache variable is updated, only the cores storing that variable's cache line need to be contacted.
-

10. Disadvantage of Directory-Based Cache Coherence

The main disadvantage mentioned in the notes is that **substantial additional storage is required for the directory**.

Thus, directory-based coherence trades additional directory storage for reduced broadcast communication.

11. Comparison: Snooping vs Directory-Based Cache Coherence

Feature	Snooping Cache Coherence	Directory-Based Cache Coherence
Basic idea	Cores observe broadcasts on the interconnect	Directory records the status of cache lines
Communication	Broadcast	Targeted communication
Update handling	Update information is broadcast to other cores	Directory identifies cores having the relevant cache line
Cache invalidation	Other cores snoop the update and invalidate their copy	Cache controllers of identified cores invalidate their lines
Scalability	Not scalable for large systems	Designed to address the scalability problem of snooping
Main overhead	Broadcast traffic	Additional directory storage
Suitable system size	More suitable when broadcast is manageable	Better suited to large systems
Write-through	Works with write-through caches	—
Write-back	Requires additional communication	—

12. False Sharing : Related Cache-Coherence Issue

An important related issue discussed immediately after directory-based coherence is **false sharing**.

CPU caches operate on **cache lines rather than individual variables**.

Consequently, two cores may modify different variables that happen to be located in the same cache line.

This can cause repeated cache-line invalidation even though the cores are not actually accessing each other's variables.

Effect

False sharing does **not cause incorrect results**, but it can seriously degrade performance by causing unnecessary accesses to main memory.

The notes state that its effect can be reduced by using **temporary storage local to the thread or process** and subsequently copying the temporary storage to shared storage.

Question 3

Explain nondeterminism and race conditions in shared-memory parallel programs. How can race conditions be avoided?

Answer

1. Introduction

In shared-memory parallel programs, multiple threads execute independently and communicate through **shared variables**. Since the threads may execute at different rates, the order in which their statements complete cannot always be predicted. This can lead to **nondeterminism** and, when simultaneous access to a shared resource can produce an error, a **race condition**.

2. Nondeterminism

A computation is said to be **nondeterministic if a given input can result in different outputs**. In a MIMD system, processors execute asynchronously, so

nondeterminism is likely to occur.

When multiple threads execute independently:

- Their relative execution rates can vary from one execution to another.
- The order in which statements complete cannot be predicted.
- Consequently, the program may produce different results from run to run.

Example of nondeterminism

Consider two threads:

```
Thread 0: my_x = 7  
Thread 1: my_x = 19
```

Both execute an output statement. The output may appear as:

```
Thread 0 > my_x = 7  
Thread 1 > my_x = 19
```

or:

```
Thread 1 > my_x = 19  
Thread 0 > my_x = 7
```

The order changes because the threads execute independently.

Nondeterminism is not necessarily an error. It becomes particularly problematic in shared-memory programs when different execution orders can produce incorrect results.

3. Race Condition

A **race condition** occurs when threads or processes attempt to simultaneously access a shared resource and those accesses can result in an error. The threads

are said to be in a "race" because the outcome depends on **which thread wins the race**.

Thus:

```
Nondeterministic execution
    |
    v
Different possible execution orders
    |
    v
Simultaneous access to shared resource
    |
    v
If access can cause an error
    |
    v
Race condition
```

4. Example of a Race Condition

Suppose two threads each compute a private value `my_val` and add it to a shared-memory location `x`.

Initially:

```
x = 0
```

Each thread executes:

```
my_val = Compute_val(my_rank);
x += my_val;
```

Assume:

```
Thread 0: my_val = 7
Thread 1: my_val = 19
```

The expected final value is:

```
x = 0 + 7 + 19 = 26
```

However, the statement:

```
x += my_val;
```

is not simply one indivisible operation. An addition typically involves:

1. Loading the values into registers.
2. Performing the addition.
3. Storing the result back into memory.

Possible execution sequence

Core 0		Core 1	
Time			
0	Finish my_val = 7	In Compute_val	
1	Load x = 0 into register	Finish my_val = 19	
2	Load my_val = 7	Load x = 0 into register	
3	Add 7 to x	Load my_val = 19	
4	Store x = 7	Add 19 to x	
5	Start other work	Store x = 19	

The final value can therefore become:

```
x = 19
```

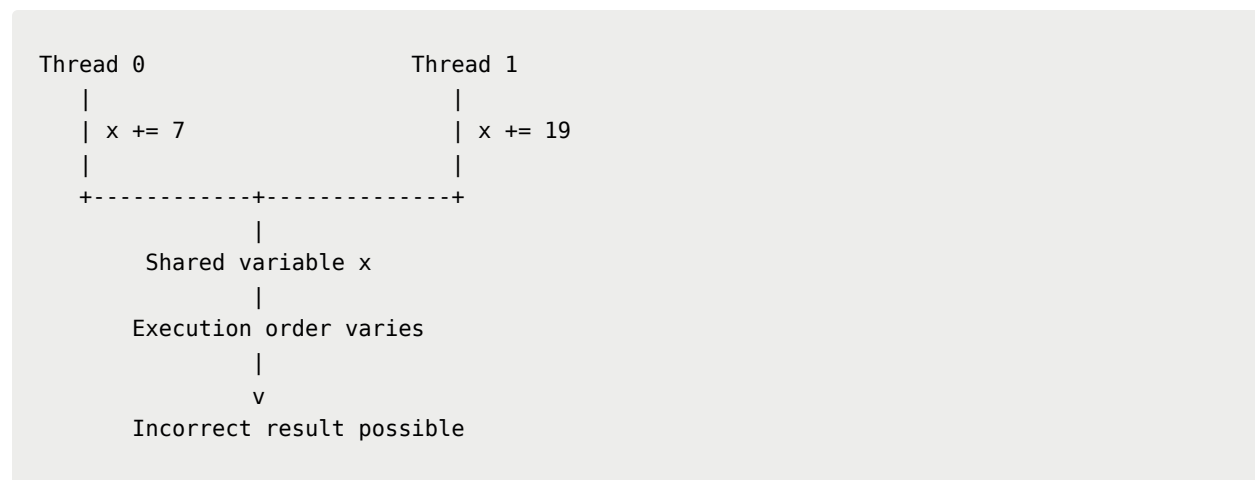
instead of the required:

```
x = 26
```

The incorrect result occurs because both threads read the same old value of x before either completed its update. The other execution sequences can also produce incorrect values.

5. Why the Race Condition Occurs

The problem is that both threads are attempting to update the same shared memory location approximately simultaneously.



The outcome depends on which thread completes the operation first. Therefore, the update operation needs to be made **atomic**.

6. Atomic Operation

An operation that writes to a memory location is **atomic** if, after a thread has completed the operation, it appears that no other thread has modified the memory location.

For the example above, the operation:

```
x += my_val;
```

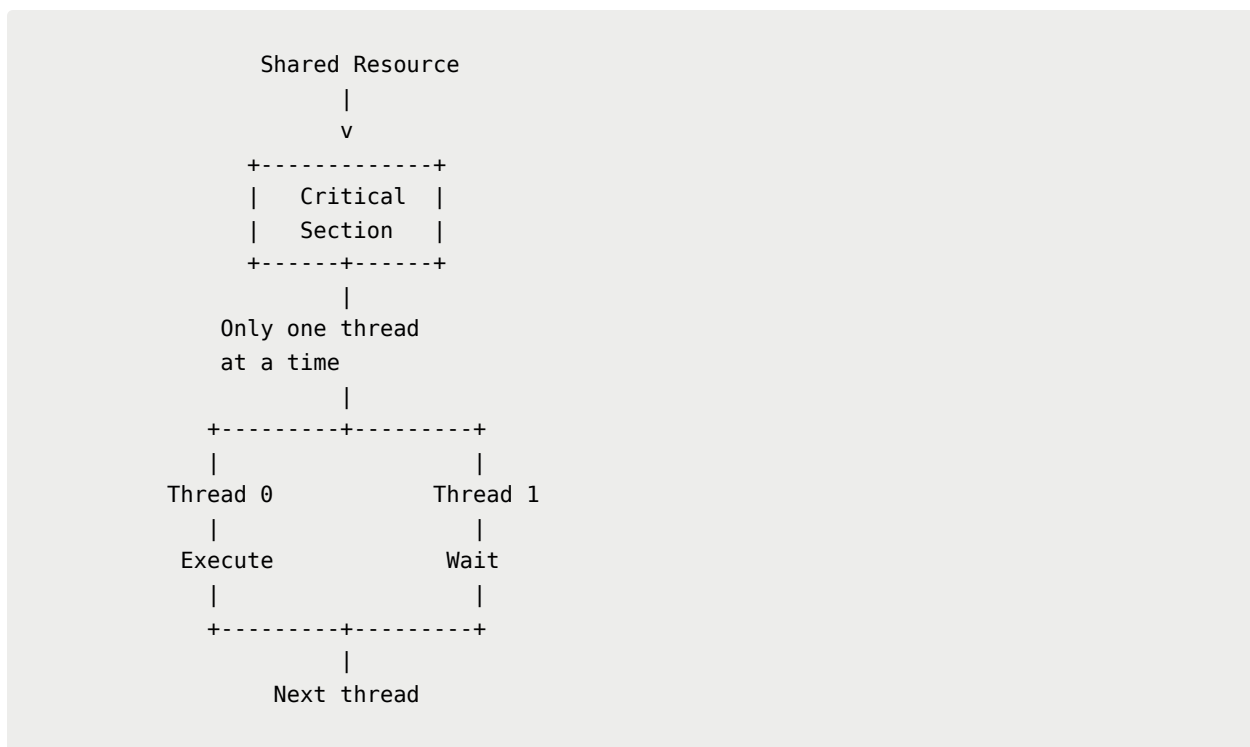
must be performed atomically.

One way of achieving this is to ensure that **only one thread executes the update at a time**.

7. Avoiding Race Conditions Using Critical Sections

A **critical section** is a block of code that can be executed by **only one thread at a time**. The programmer must ensure **mutually exclusive access** to the critical section.

Diagram



If one thread is executing the critical section, other threads are excluded until it finishes.

8. Mutex / Lock

The most commonly used mechanism for ensuring mutual exclusion is a **mutual exclusion lock (mutex)**, also called simply a **lock**.

A mutex is a special type of object with support in the underlying hardware.

Working of Mutex

```
Thread
|
v
Obtain Lock
|
v
+-----+
| Critical Section |
|   x += my_val   |
+-----+
|
v
Unlock
```

Before entering the critical section, a thread must obtain the mutex using a lock function. After completing the critical section, it relinquishes the mutex using an unlock function.

While one thread owns the lock, another thread attempting to execute the critical section waits at the lock function.

9. Corrected Program Using Mutex

We modify the previous program as follows:

```
my_val = Compute_val(my_rank);

Lock(&add_my_val_lock);

x += my_val;

Unlock(&add_my_val_lock);
```

This ensures that only **one thread at a time** can execute:

```
x += my_val;
```

Therefore, the race condition is avoided.

Important observation

The mutex does **not** impose a predetermined order on the threads. Either Thread 0 or Thread 1 may execute the critical section first. It only ensures that they do not execute it simultaneously.

10. Serialization of the Critical Section

The use of a mutex causes the critical section to be **serialized**.

Since only one thread can execute the critical section at a time, that portion of the program is effectively serial. Therefore:

- The number of critical sections should be minimized.
- Critical sections should be kept as short as possible.

This is important because excessive use of locks can reduce the parallelism of the program.

11. Alternative: Busy-Waiting

Another method is **busy-waiting**.

In busy-waiting, a thread repeatedly tests a condition until it is allowed to enter the critical section.

For example:

```
my_val = Compute_val(my_rank);

if (my_rank == 1)
    while (!ok_for_1);          /* Busy-wait loop */

x += my_val;                    /* Critical section */
```

```
if (my_rank == 0)
    ok_for_1 = true;
```

Here, `ok_for_1` is initially `false`. Thread 1 remains in the busy-wait loop until Thread 0 sets:

```
ok_for_1 = true;
```

After that, Thread 1 can update `x`.

Disadvantage of Busy-Waiting

Busy-waiting is simple to understand and implement, but it can be **very wasteful of system resources**. Even though a thread is doing no useful work, the core repeatedly checks whether the critical section can be entered.

12. Other Synchronization Mechanisms

Semaphores

Semaphores are similar to mutexes, although their detailed behavior is different. Some types of thread synchronization are easier to implement using semaphores than mutexes.

Monitor

A **monitor** provides mutual exclusion at a higher level. It is an object whose methods can only be executed by one thread at a time.

13. Thread Safety and Race Conditions

Thread safety is also important in shared-memory programs.

Ordinary local variables declared inside a function are generally private because each thread has its own stack. However, **static local variables** are effectively shared among threads that call the function.

The notes give `strtok` as an example of a function that is **not thread safe** because it uses a static variable. If two threads use it simultaneously, one thread's string information can be lost or overwritten by the other thread.

Thus, programmers must be careful when using functions written for serial programs in multithreaded programs.

14. Methods to Avoid Race Conditions

race conditions can be avoided by:

1. **Making the required operation atomic.**
2. **Using a critical section** so that only one thread accesses the relevant shared resource at a time.
3. **Using a mutex/lock** to enforce mutual exclusion.
4. **Using busy-waiting** where appropriate, while recognizing its resource cost.
5. **Using semaphores** for appropriate synchronization requirements.
6. **Using monitors**, which provide mutual exclusion at a higher level.
7. Being careful with **non-thread-safe functions**, particularly functions using shared/static data.

15. Nondeterminism vs Race Condition

Aspect	Nondeterminism	Race Condition
Meaning	Same input can result in different outputs	Simultaneous access to a shared resource can result in an error
Cause	Independent threads execute at varying rates	Threads race to access/update a shared resource
Result	Different execution orders/results may occur	Incorrect computation can occur
Always an error?	No	Yes, when the competing accesses can produce an error

Aspect	Nondeterminism	Race Condition
Example	Different order of thread output	Two threads simultaneously execute <code>x += my_val</code>
Solution	Synchronization when deterministic ordering/correct coordination is required	Atomic operations, critical sections, mutexes, and other synchronization mechanisms

Nondeterminism is not always a problem, whereas nondeterminism in shared-memory programs can be disastrous when it results in program errors.

Question 4

Explain SIMD architecture and data parallelism with a neat diagram and suitable example. Discuss its advantages and limitations.

Answer

1. Introduction

SIMD (Single Instruction, Multiple Data) is a parallel architecture in which a **single instruction stream operates on multiple data items simultaneously**.

An abstract SIMD system consists of a **single control unit and multiple datapaths**. The control unit broadcasts an instruction to the datapaths, and each datapath applies the instruction to its current data item or remains idle.

SIMD systems are particularly suitable for **simple loops that operate on large arrays of data**.

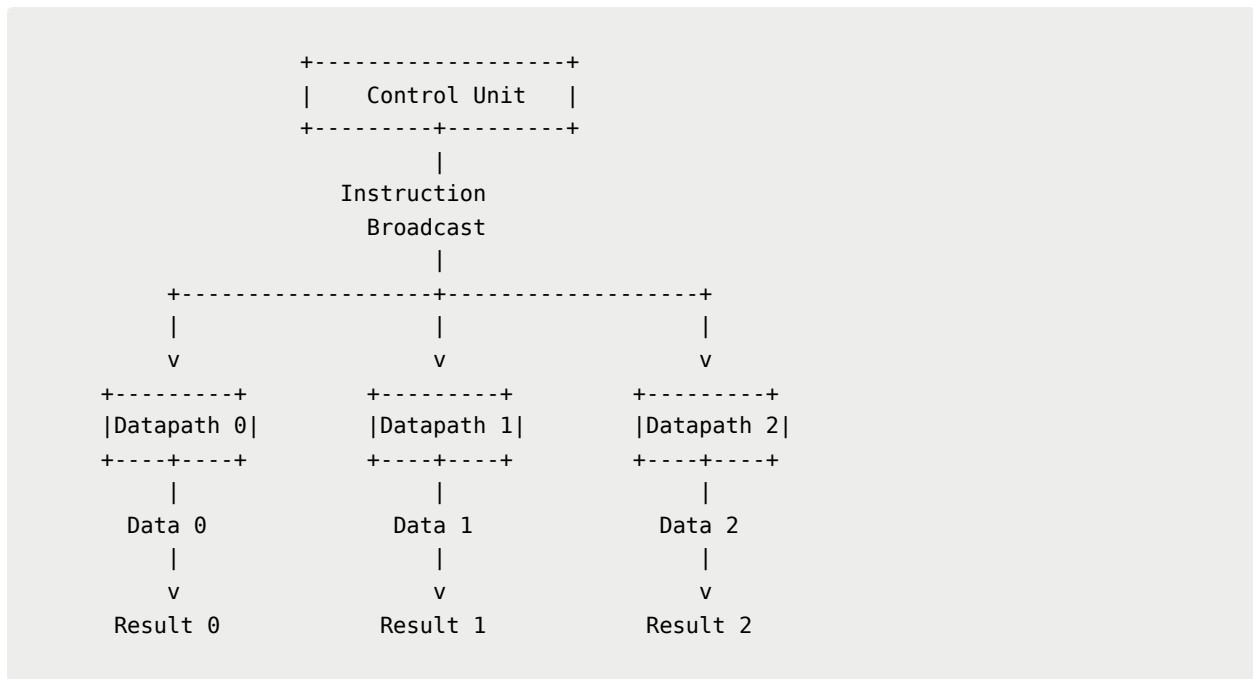
2. SIMD Architecture

The basic SIMD architecture contains:

- **Single Control Unit**
- **Multiple Datapaths**
- Multiple data items

- A mechanism for broadcasting the same instruction to the datapaths

Neat Diagram



Working

1. The **control unit** generates an instruction.
2. The instruction is **broadcast** to all datapaths.
3. Each datapath receives its corresponding data item.
4. Each datapath applies the same instruction to its data item.
5. If a datapath does not need to perform the operation, it remains idle.

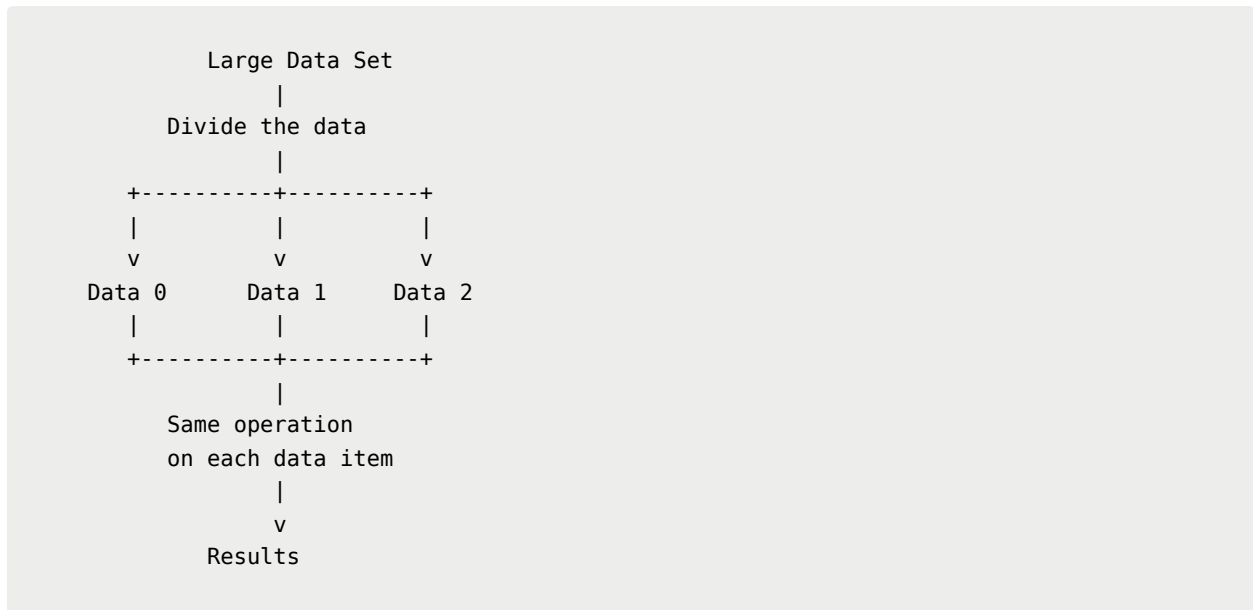
In a classical SIMD system, the datapaths operate **synchronously**. Each datapath waits for the next instruction to be broadcast before proceeding. Datapaths do not have instruction storage.

3. Data Parallelism

Data parallelism is the parallelism obtained by **dividing data among processors** and having the processors apply approximately the same instructions to their

respective subsets of data.

Thus, the basic idea is:



SIMD is therefore well suited to data-parallel problems because the same operation can be applied to many data items.

4. Suitable Example — Vector Addition

Consider two arrays `x` and `y`, each containing `n` elements. The objective is to add the elements of `y` to the corresponding elements of `x`.

The serial operation is:

```
for (i = 0; i < n; i++)
    x[i] += y[i];
```

Suppose the SIMD system has `n` datapaths.

Data:

<code>x[0]</code>	<code>x[1]</code>	<code>x[2]</code>	<code>...</code>	<code>x[n-1]</code>
<code>+</code>	<code>+</code>	<code>+</code>		<code>+</code>
<code>y[0]</code>	<code>y[1]</code>	<code>y[2]</code>	<code>...</code>	<code>y[n-1]</code>
<code> </code>	<code> </code>	<code> </code>		<code> </code>

```
      v      v      v      ...      v
    x[0]   x[1]   x[2]   ...   x[n-1]
```

The elements `x[i]` and `y[i]` are loaded into the corresponding datapaths. Each datapath performs the same addition:

```
x[i] = x[i] + y[i]
```

and stores the result in `x[i]`.

Example with fewer datapaths

If there are `m` datapaths and `m < n`, the additions are performed in blocks of `m` elements.

For example:

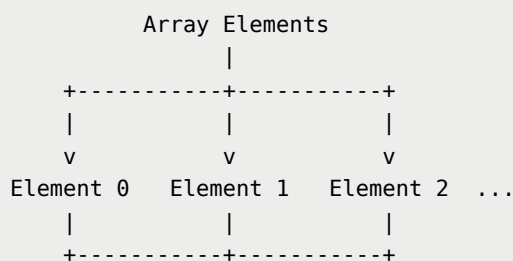
```
Number of datapaths = 4
Number of elements  = 15

Block 1 → elements 0-3
Block 2 → elements 4-7
Block 3 → elements 8-11
Block 4 → elements 12-14
```

In the last block, only three elements are processed, so **one of the four datapaths remains idle**.

5. Data Parallelism in SIMD

The vector addition example demonstrates data parallelism:



```

      |
Same operation
x[i] += y[i]
      |
      v
New values

```

The **data is divided**, while the **same operation is applied** to each subset of data. This is the fundamental form of parallelism supported by SIMD.

6. Advantages of SIMD

1. Efficient data-parallel processing

SIMD can be **very efficient on large data-parallel problems**, particularly when the same operation needs to be applied to many data items.

2. Suitable for large arrays

SIMD systems are ideal for **simple loops operating on large arrays of data**.

3. Simultaneous operation on multiple data items

A single instruction can be applied to multiple data items through multiple datapaths.

4. Vector processing

Vector processors can operate on arrays or vectors of data rather than individual scalar data elements.

5. Efficient vector instructions

For vector systems, a simple loop such as:

```
for (i = 0; i < n; i++)
    x[i] += y[i];
```

can require only a single load, add, and store for each block of vector-length elements, whereas a conventional system requires a load, add, and store for each

individual element.

7. Limitations of SIMD

1. Different execution paths reduce performance

All datapaths must execute the same instruction or remain idle. This can seriously reduce SIMD performance when different data items require different operations.

For example:

```
for (i = 0; i < n; i++)  
    if (y[i] > 0.0)  
        x[i] += y[i];
```

If `y[i]` is not positive, the corresponding datapath remains idle while other datapaths perform the addition.

```
Data 0 → condition TRUE → ADD  
Data 1 → condition TRUE → ADD  
Data 2 → condition FALSE → IDLE  
Data 3 → condition TRUE → ADD
```

2. Synchronization of datapaths

In a classical SIMD system, datapaths must operate **synchronously**. Each datapath waits for the next instruction broadcast before continuing.

3. Limited suitability for irregular problems

SIMD systems are highly effective for large data-parallel problems but often do not perform well on other types of parallel problems.

4. Idle datapaths

When the number of data elements is not an exact multiple of the number of datapaths, some datapaths may remain idle. In the example with 15 elements and 4 datapaths, one datapath is idle during the final block.

5. Poor performance on small problems

In the context of GPUs using SIMD parallelism, that many threads processing large amounts of data are needed to keep the datapaths busy and GPUs may have relatively poor performance on small problems.

8. Applications / Examples of SIMD

Vector Processors

Vector processors operate on **arrays or vectors of data**, whereas conventional CPUs operate on individual data elements or scalars.

GPUs

GPUs use SIMD parallelism to process multiple elements in graphics streams. Current-generation GPUs use SIMD parallelism and obtain it by including a large number of datapaths on each GPU processing core.

Desktop CPUs

Desktop CPUs also make use of aspects of SIMD computing.

Note: GPUs are **not pure SIMD systems**; they can run multiple instruction streams and independent cores.

9. SIMD Architecture vs Data Parallelism

SIMD Architecture	Data Parallelism
Hardware/architecture model	Form of parallelism
Uses a single instruction stream	Divides data among processors
Has a single control unit and multiple datapaths	Same or approximately same instructions are applied to data subsets
Instruction is broadcast to datapaths	Data is divided among processors
Datapaths operate on multiple data items	Each processor operates on its assigned data

SIMD Architecture	Data Parallelism
Example: vector processor	Example: dividing array elements among processors and applying the same operation

Question 5

Explain MIMD systems, their characteristics and asynchronous execution. Give suitable examples.

Answer

1. Introduction

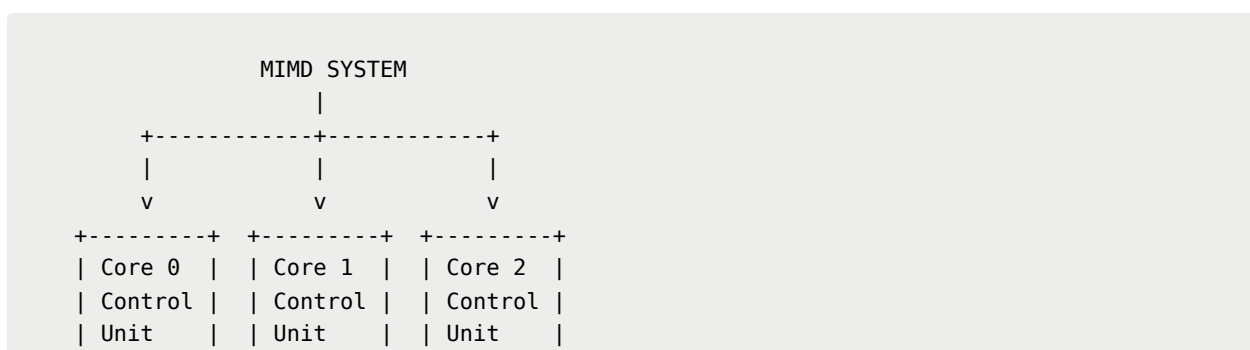
MIMD (Multiple Instruction, Multiple Data) systems support **multiple simultaneous instruction streams operating on multiple data streams**. They typically consist of a collection of independent processing units or cores, where each core has its own **control unit and datapath**.

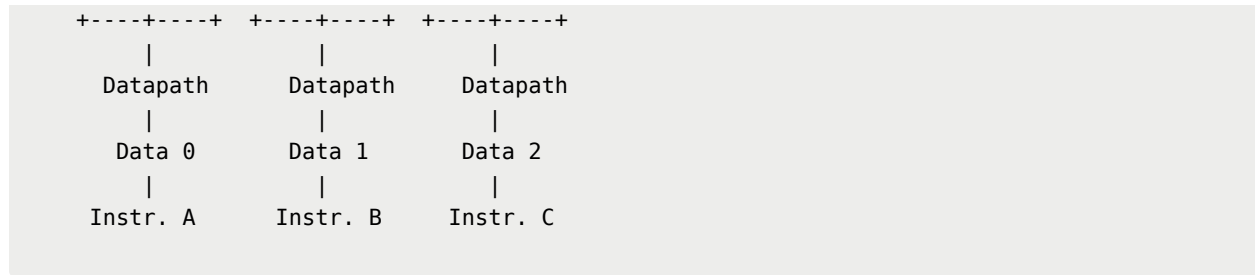
MIMD systems are an important class in **Flynn's taxonomy**, which classifies parallel computers according to the number of instruction streams and data streams they can simultaneously manage.

2. MIMD Architecture

In a MIMD system, each processing unit can execute its own instruction stream on its own data.

Neat Diagram





Unlike SIMD, where one instruction is broadcast to multiple datapaths, MIMD allows different processing units to execute **different instructions on different data**.

3. Characteristics of MIMD Systems

1. Multiple instruction streams

MIMD systems support **multiple simultaneous instruction streams**. Different processing units can therefore execute different instructions.

2. Multiple data streams

The different instruction streams operate on **multiple data streams**. Thus, different processing units can work on different data.

3. Independent processing units

MIMD systems typically consist of a collection of **fully independent processing units or cores**. Each processing unit has its own control unit and datapath.

4. Asynchronous operation

Unlike SIMD systems, MIMD systems are usually **asynchronous**. The processors can operate at their own pace.

5. Shared-memory or distributed-memory organization

There are two principal types of MIMD systems:

- **Shared-memory systems**
- **Distributed-memory systems**

6. Communication differs according to memory organization

In a **shared-memory system**, processors communicate implicitly by accessing shared data structures.

In a **distributed-memory system**, each processor has its own private memory and processors communicate explicitly through an interconnection network, usually by sending messages or using special functions that provide access to another processor's memory.

4. Asynchronous Execution in MIMD

Definition

MIMD systems are usually **asynchronous**, meaning that processors can operate at their **own pace**.

In many MIMD systems:

- There is **no global clock**.
- There may be no relationship between the system times of two different processors.
- Different processors can execute different statements at the same instant.
- Even if processors execute exactly the same sequence of instructions, they may be at different points in that sequence at any given instant.

Diagram

Time →

Processor 0:
Instr 1 → Instr 2 → Instr 3 → Instr 4

Processor 1:
Instr 1 → Instr 2 → Instr 3

Processor 2:
Instr 1 → Instr 2 → Instr 3 → Instr 4 → ...

Each processor operates at its own pace

Therefore, there is no requirement that all processors reach the same instruction at the same time.

5. Effect of Asynchronous Execution

Asynchronous execution can result in **nondeterminism**.

A computation is nondeterministic if a given input can result in different outputs. Since independently executing threads can complete their statements at different rates from one execution to another, the results of the program may differ from run to run.

For example, consider two threads:

```
Thread 0: my_x = 7  
Thread 1: my_x = 19
```

Both execute an output statement.

One possible output order is:

```
Thread 0 > my_x = 7  
Thread 1 > my_x = 19
```

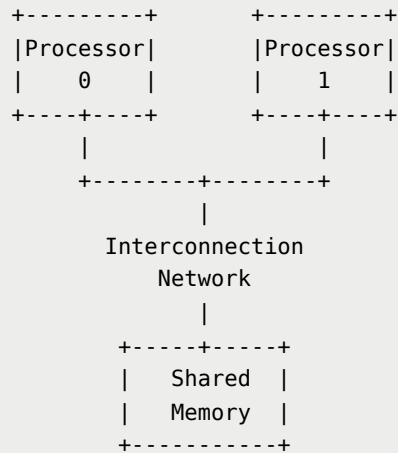
Another possible order is:

```
Thread 1 > my_x = 19  
Thread 0 > my_x = 7
```

The output order cannot be predicted because the threads execute independently.

6. Shared-Memory MIMD Systems

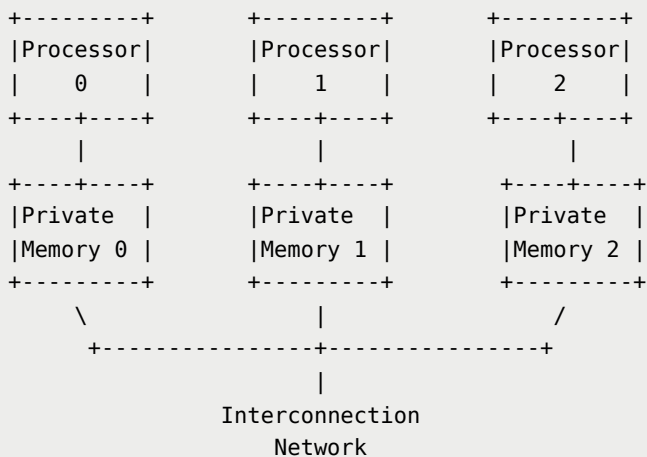
In a shared-memory MIMD system, a collection of autonomous processors is connected to a memory system through an **interconnection network**.



Each processor can access each memory location. Processors usually communicate **implicitly** by accessing shared data structures.

7. Distributed-Memory MIMD Systems

In a distributed-memory MIMD system, each processor is paired with its own **private memory**.



The processor-memory pairs communicate through an interconnection network. Communication is usually explicit through **message passing** or special functions that provide access to another processor's memory.

8. Examples of MIMD Systems

Example 1: Multicore shared-memory system

A multicore system in which multiple independent cores execute different instruction streams and access shared memory represents a **shared-memory MIMD system**.

The shared-memory system is a collection of autonomous processors connected to a memory system, with processors able to access each memory location.

Example 2: Cluster

Clusters are widely available distributed-memory systems. A cluster consists of a collection of commodity systems, such as PCs, connected through a commodity interconnection network such as Ethernet.

Such systems represent distributed-memory organization in which processor-memory pairs communicate through a network.

Example 3: Grid

A **grid** provides infrastructure for turning large networks of geographically distributed computers into a unified distributed-memory system. Such systems are generally heterogeneous, meaning that individual nodes can be built from different types of hardware.

Question 6

Explain UMA and NUMA shared-memory architectures with neat diagrams. Compare their memory-access characteristics.

Answer

1. Introduction

Shared-memory systems with multiple multicore processors can be organized in two ways depending on how the processors are connected to main memory:

1. **Uniform Memory Access (UMA)**
2. **Nonuniform Memory Access (NUMA)**

In UMA, the time to access memory locations is the same for all cores. In NUMA, a core can access its directly connected memory more quickly than memory connected to another processor/chip.

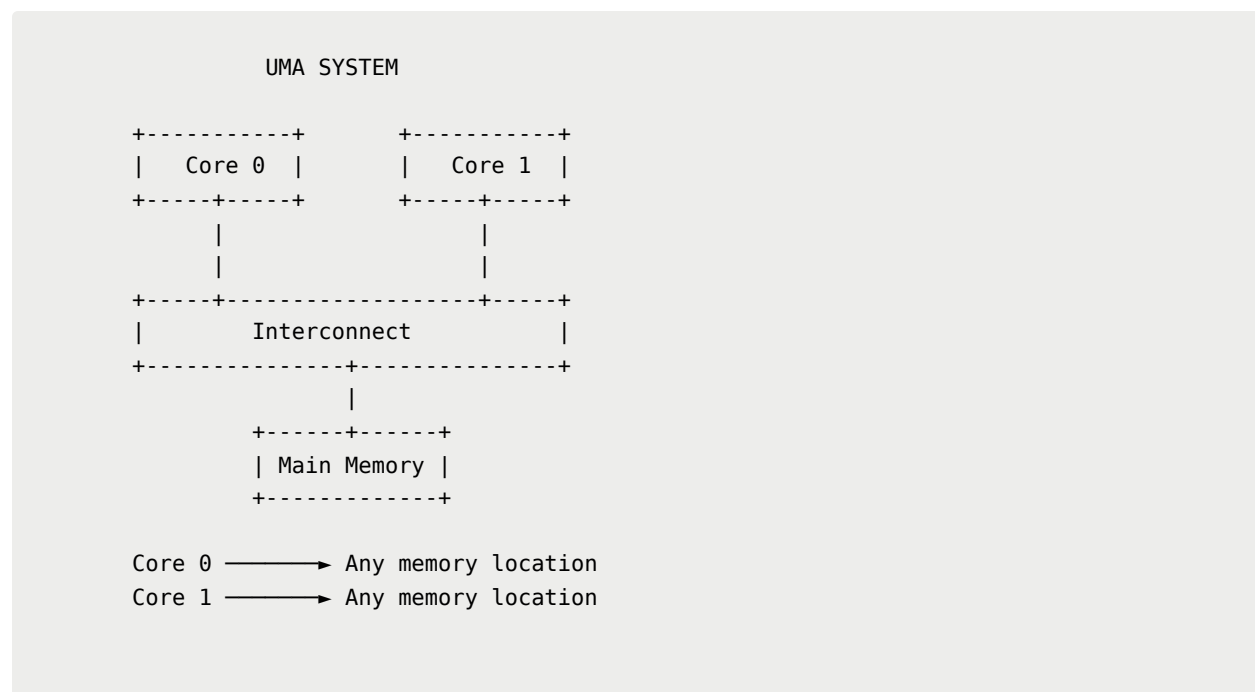
2. Uniform Memory Access (UMA)

Definition

A **Uniform Memory Access (UMA)** system is a shared-memory architecture in which **all processors have the same access time to all memory locations**.

In this organization, the interconnect connects all processors directly to the main memory.

UMA Architecture



Access time = Same

Explanation: In UMA, the processors are connected to the common main memory through an interconnection network. Since the memory is uniformly accessible, the time required for accessing memory locations is the same for all cores.

3. Characteristics of UMA

1. All processors share the same main memory.
2. All cores have the **same memory-access time** to memory locations.
3. Processors are connected to main memory through an interconnection network.
4. The programmer does not need to worry about different access times for different memory locations.
5. UMA systems are therefore comparatively easier to program.

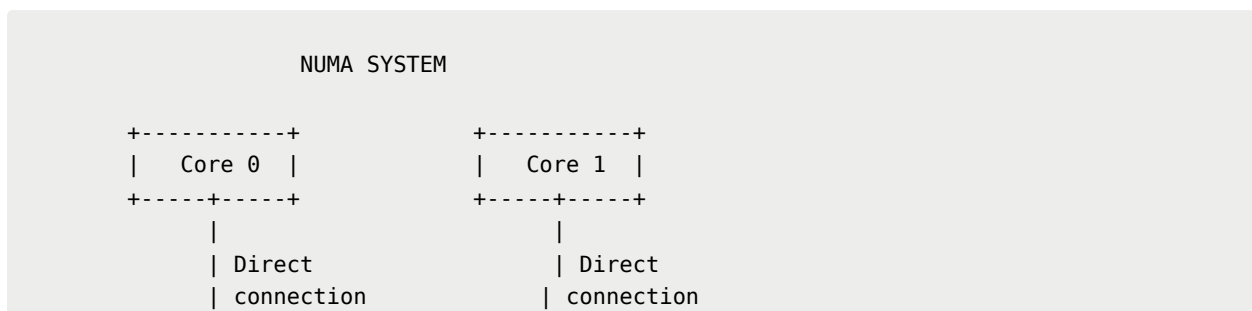
4. Nonuniform Memory Access (NUMA)

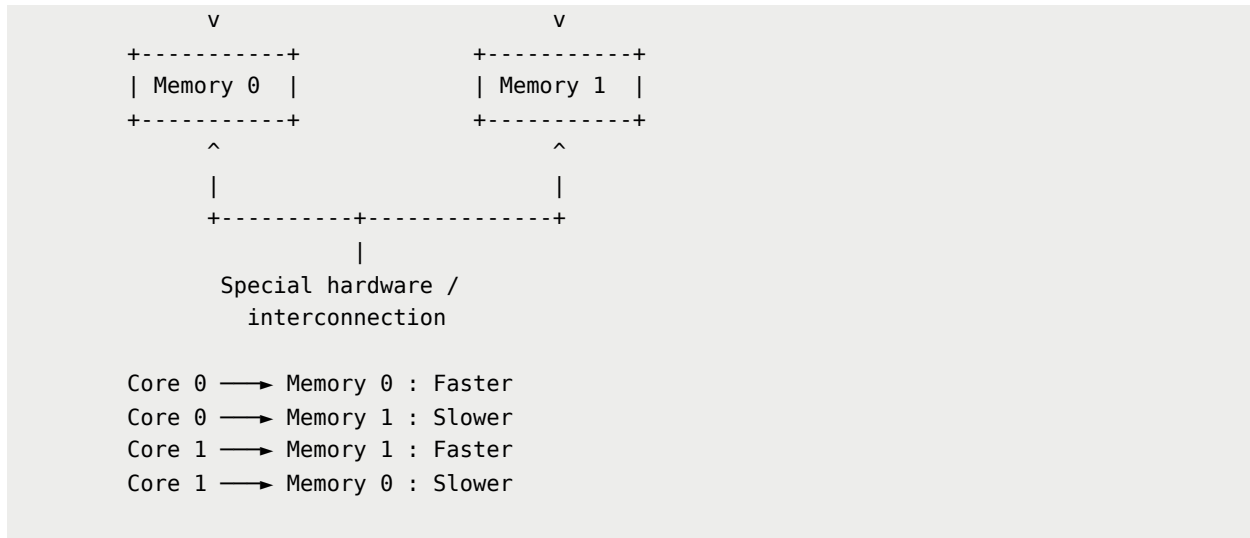
Definition

A **Nonuniform Memory Access (NUMA)** system is a shared-memory architecture in which **each processor has a direct connection to a block of main memory**.

A processor can also access the memory block connected to another processor through special hardware built into the processors.

NUMA Architecture





Explanation: Each processor has a direct connection to its own block of main memory. It can also access another processor's memory through special hardware. Therefore, accessing directly connected memory is faster than accessing memory connected to another chip.

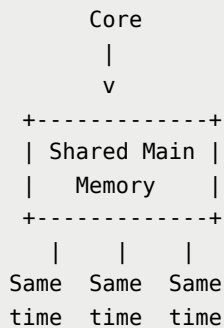
5. Characteristics of NUMA

1. Each processor has a **direct connection to a block of main memory**.
2. A processor can access another processor's memory through special hardware.
3. Access time depends on the location of the memory being accessed.
4. Directly connected memory can be accessed **more quickly**.
5. Memory that must be accessed through another chip takes longer to access.
6. NUMA systems have the potential to use **larger amounts of memory than UMA systems**.

6. Memory-Access Characteristics

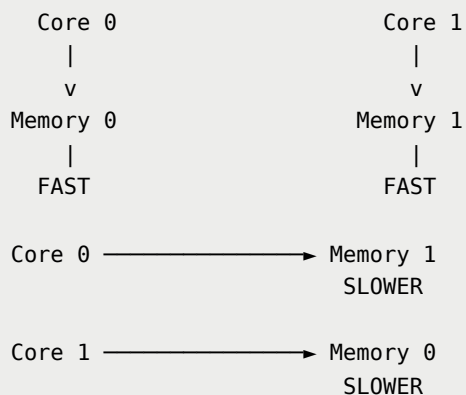
The fundamental difference between UMA and NUMA is the **uniformity of memory-access time**.

UMA



In UMA, all memory locations take the **same amount of time to access**, regardless of which core performs the access.

NUMA



In NUMA, the memory location directly connected to a core can be accessed more quickly than a memory location that must be accessed through another chip.

7. UMA vs NUMA Comparison

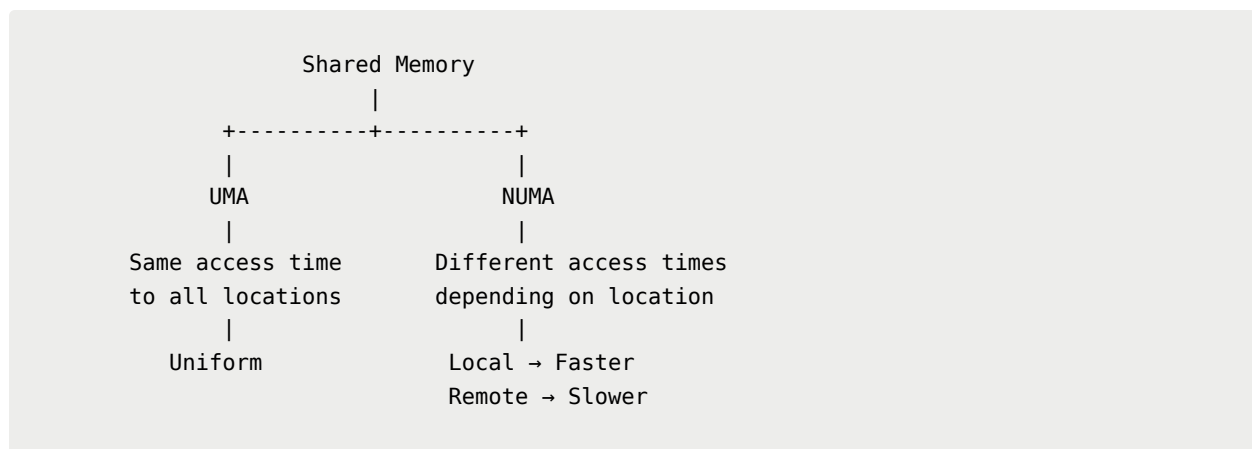
Feature	UMA	NUMA
Full form	Uniform Memory Access	Nonuniform Memory Access
Memory organization	Common main memory	Memory divided into blocks associated with processors

Feature	UMA	NUMA
Processor connection	Processors are connected directly to main memory through the interconnect	Each processor has a direct connection to a block of main memory
Access to local/direct memory	Same as other memory accesses	Faster
Access to another processor's memory	Same access time	Slower
Memory-access time	Uniform	Nonuniform
Programmer concern	Does not need to worry about different memory-access times	Memory location can affect access time
Programming	Generally easier	More complex because access times differ
Memory capacity	Less potential compared with NUMA according to the notes	Has potential to use larger amounts of memory

The UMA is easier to program because the programmer does not need to consider different access times, while NUMA can provide faster access to directly connected memory and has the potential to use larger amounts of memory.

8. Key Difference in Memory Access

The difference can be summarized as:



Therefore, the terms **uniform** and **nonuniform** directly describe the memory-access behavior:

- **UMA:** Access time is uniform.
 - **NUMA:** Access time is dependent on the memory's location relative to the processor.
-

Question 7

Explain the interconnection networks used in parallel computers and discuss their effect on system performance.

Answer

1. Introduction

An **interconnection network (interconnect)** provides communication between processors, memory modules, and other components of a parallel computer.

The interconnect plays a **decisive role in the performance** of both shared-memory and distributed-memory systems. Even if processors and memory have very high performance, a slow interconnect can seriously degrade the overall performance of a parallel program.

2. Interconnection Networks

Interconnection networks can be considered separately for:

1. **Shared-memory systems**
2. **Distributed-memory systems**

The buses and switched interconnects for shared-memory systems, and **direct and indirect interconnects** for distributed-memory systems.

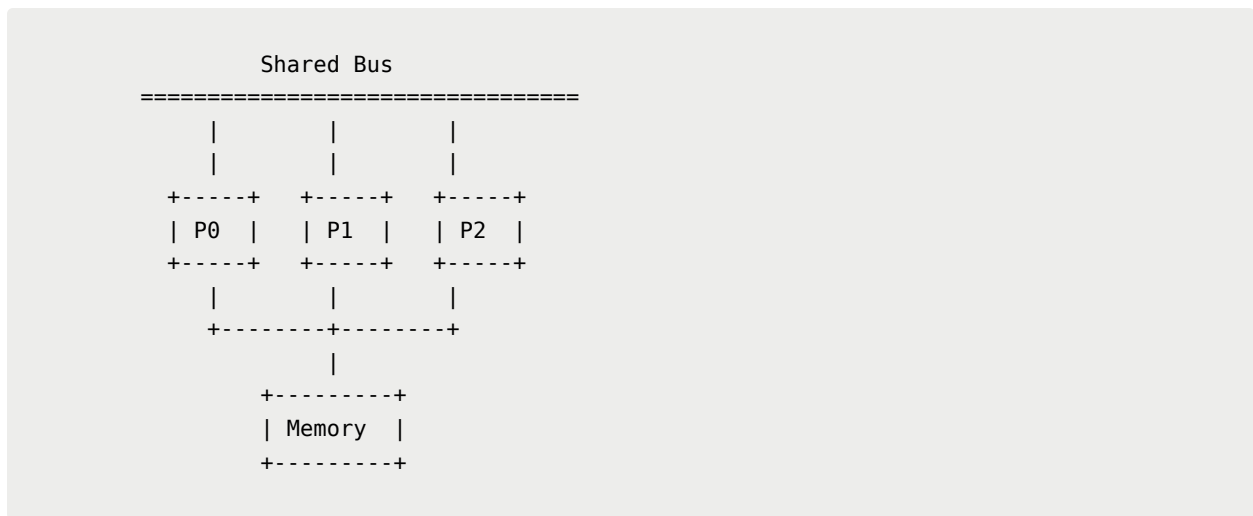
3. Shared-Memory Interconnects

3.1 Bus

A **bus** is a collection of parallel communication wires together with hardware that controls access to the bus.

The key characteristic is that the communication wires are **shared by all devices connected to the bus**.

Diagram



Advantages of Bus

- **Low cost**
- **Flexible**
- Multiple devices can be connected with little additional cost.

Limitation of Bus

Since the communication wires are shared, increasing the number of connected devices increases the likelihood of **contention** for the bus.

As a result:

```
More processors
  ↓
More contention
  ↓
More waiting
```

↓
Lower performance

With a large number of processors, processors may frequently have to wait for access to main memory. Therefore, buses are suitable for systems with only a few processors.

4. Switched Interconnects

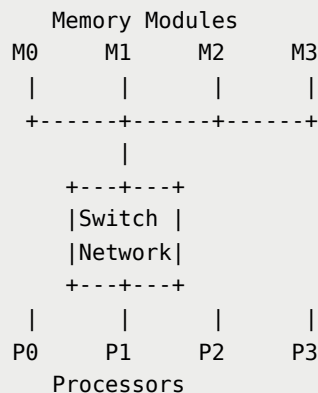
As shared-memory systems have increased in size, buses have increasingly been replaced by **switched interconnects**.

A switched interconnect uses **switches to control the routing of data among connected devices**.

A **crossbar** is a relatively simple and powerful switched interconnect.

4.1 Crossbar Interconnect

Diagram



The crossbar contains switches that control the connections between processors and memory modules.

If there are at least as many memory modules as processors, a conflict occurs only when two cores attempt to access the **same memory module simultaneously**.

Performance

Crossbars allow **simultaneous communication among different devices**. Therefore, they are much faster than buses.

Limitation

The switches and links are relatively expensive. Thus, a small bus-based system is much less expensive than a crossbar-based system of the same size.

5. Distributed-Memory Interconnects

Distributed-memory interconnects are divided into two groups:

1. **Direct interconnects**
 2. **Indirect interconnects**
-

6. Direct Interconnects

In a **direct interconnect**, each switch is directly connected to a processor-memory pair, and the switches are connected to one another.

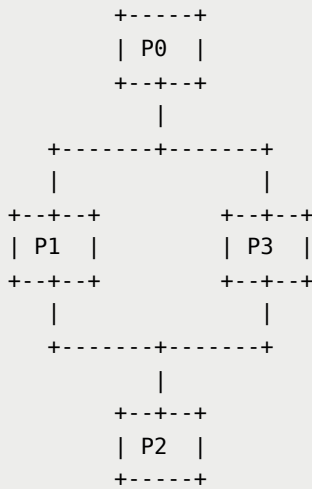
Examples include:

- Ring
 - Toroidal mesh
 - Fully connected network
 - Hypercube
-

6.1 Ring

A ring is a direct interconnect in which processors/switches are connected in a circular arrangement.

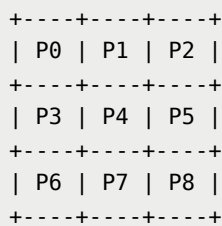
Diagram



A ring is superior to a simple bus because it allows **multiple simultaneous communications**. However, communication patterns can occur in which some processors must wait for other processors to complete their communications.

6.2 Toroidal Mesh

A toroidal mesh is another direct interconnect.



Horizontal and vertical
connections wrap around

The toroidal mesh is more expensive than the ring because its switches must support more links. However, it provides a **greater number of possible simultaneous communication patterns** than a ring.

7. Connectivity and Bisection Width

One measure of the number of simultaneous communications, or **connectivity**, is the **bisection width**.

Definition

Imagine dividing a parallel system into two groups of equal size. The **bisection width** measures how many simultaneous communications can take place across the divide.

Alternatively, it can be determined as the minimum number of links that must be removed to divide the nodes into two equal groups that cannot communicate with one another.

For a ring with eight nodes, the bisection width is **2**.

For a square two-dimensional toroidal mesh with $p = q^2$ nodes, the bisection width is:

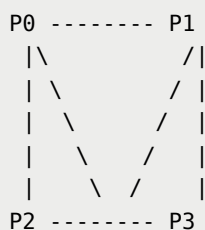
$$2q = 2\sqrt{p}$$

A larger bisection width indicates greater potential connectivity between the two halves of the system.

8. Fully Connected Network

The **fully connected network** is a highly connected direct interconnect in which every switch is directly connected to every other switch.

Diagram



It provides very high connectivity, but it is **impractical for systems with more than a few nodes** because it requires a very large number of links.

9. Hypercube

A **hypercube** is a highly connected direct interconnect that has been used in actual systems.

A hypercube is constructed inductively:

- A 1-dimensional hypercube has 2 processors.
- A 2-dimensional hypercube is constructed from two 1-dimensional hypercubes.
- A 3-dimensional hypercube is constructed from two 2-dimensional hypercubes.

In general, a hypercube of dimension d has:

$$p = 2^d$$

nodes.

Its bisection width is:

$$p/2$$

and therefore it has greater connectivity than a ring or toroidal mesh. However, it is more expensive to construct than a toroidal mesh.

10. Indirect Interconnects

In an **indirect interconnect**, the switches may not be directly connected to a processor.

They are commonly shown with:

- A collection of processors

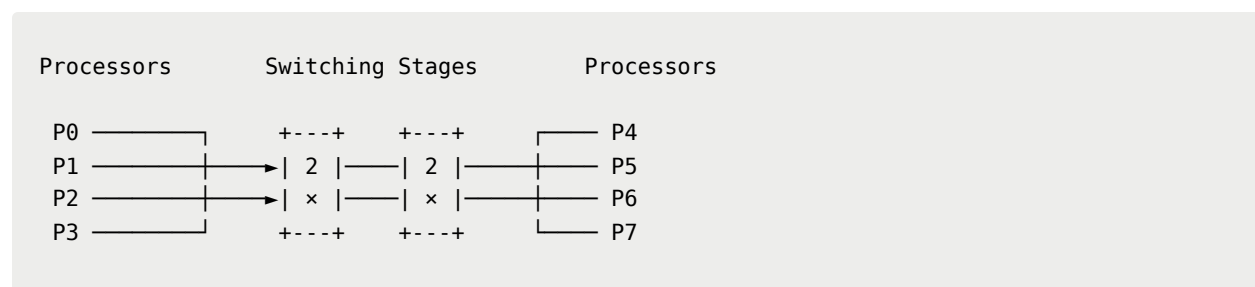
- Incoming and outgoing links
- A switching network

The **crossbar and omega network** are examples of indirect networks.

10.1 Omega Network

An omega network uses **2×2 crossbar switches**.

Basic representation



Unlike a crossbar, some communications cannot occur simultaneously. For example, if processor 0 sends a message to processor 6, processor 1 cannot simultaneously send a message to processor 7.

Advantage

The omega network is **less expensive than the crossbar**.

The notes state that an omega network uses fewer switches than a crossbar, making it less expensive.

11. Latency and Bandwidth

Two important measures used to describe interconnect performance are:

1. **Latency**
2. **Bandwidth**

Latency

Latency is the time that elapses between the **source beginning to transmit data** and the **destination beginning to receive the first byte**.

Bandwidth

Bandwidth is the rate at which the destination receives data **after it has started receiving the first byte**.

If:

- l = latency in seconds
- b = bandwidth in bytes/second
- n = message size in bytes

then the message transmission time is:

$$\text{Message transmission time} = l + n/b$$

12. Effect of Interconnection Networks on System Performance

The interconnect affects parallel-system performance in several ways.

1. Communication delay

A high-latency interconnect increases the time required for processors to communicate or for data to reach its destination.

2. Data-transfer rate

Higher bandwidth allows data to be transferred at a higher rate after transmission begins.

3. Contention

In a bus, all devices share the communication wires. As the number of processors increases, contention increases and performance decreases.

4. Simultaneous communication

Networks with greater connectivity allow more communications to occur simultaneously.

For example:

- A ring allows multiple simultaneous communications but may force some processors to wait.
- A toroidal mesh supports more simultaneous communication patterns than a ring.
- A crossbar allows simultaneous communication among different devices.

5. Scalability

The cost and performance of an interconnect influence how many processors can be effectively supported.

Large crossbars are expensive, while distributed-memory interconnects such as **hypercubes and toroidal meshes** are relatively inexpensive. The notes state that distributed-memory systems with **thousands of processors** have been built using these and other interconnects.

Question 8

Explain nondeterminism, race conditions, atomic operations and critical sections. Discuss the use of mutex locks for achieving mutual exclusion.

Answer

1. Introduction

In a shared-memory parallel program, multiple threads may execute concurrently and access shared variables. Since the threads execute independently, the order in which their statements complete may vary from one execution to another. This can lead to **nondeterminism** and, in certain cases, **race conditions**.

To obtain correct results, operations on shared data may need to be made **atomic** and protected inside **critical sections** using mechanisms such as **mutex locks**.

2. Nondeterminism

Definition

A computation is **nondeterministic** if a given input can result in **different outputs**.

In a MIMD system, processors execute asynchronously. When multiple threads execute independently, the relative rate at which they complete their statements can vary from one execution to another. Consequently, the results or order of execution may differ between executions.

Example

Consider two threads:

```
Thread 0:  
my_x = 7
```

```
Thread 1:  
my_x = 19
```

Both threads execute an output statement.

One possible output is:

```
Thread 0 > my_x = 7  
Thread 1 > my_x = 19
```

Another possible output is:

```
Thread 1 > my_x = 19  
Thread 0 > my_x = 7
```

The order cannot be predicted because the threads execute independently.

Nondeterminism is not necessarily an error. However, in shared-memory programs it can become serious when it produces incorrect results.

3. Race Condition

Definition

When threads or processes attempt to **simultaneously access a shared resource**, and these accesses can result in an error, the program is said to have a **race condition**.

The outcome depends on **which thread wins the race** to perform the operation.

Example

Suppose two threads calculate private values `my_val` and both want to update a shared variable `x`:

```
my_val = Compute_val(my_rank);  
x += my_val;
```

Assume:

```
x = 0  
  
Thread 0: my_val = 7  
Thread 1: my_val = 19
```

The statement:

```
x += my_val;
```

involves loading `x`, performing the addition, and storing the result.

A possible execution is:

Time	Thread 0	Thread 1
0	Finish assignment	In Compute_val
1	Load x = 0	Finish assignment
2	Load my_val = 7	Load x = 0
3	Add 7 to x	Load my_val = 19

4	Store x = 7	Add 19 to x
5		Store x = 19

The final value can therefore be incorrect because both threads are attempting to update the same shared memory location.

Key point

```
Concurrent access to shared resource
      ↓
Different execution order
      ↓
Possible incorrect result
      ↓
Race condition
```

4. Atomic Operations

Definition

An operation that writes to a memory location is **atomic** if, after a thread has completed the operation, it appears that **no other thread has modified the memory location**.

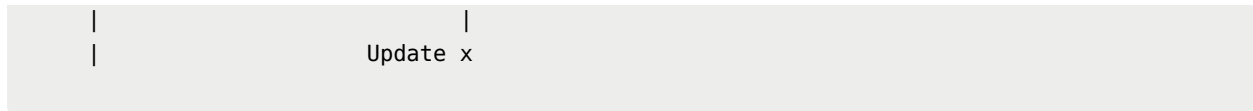
For the previous example, the operation:

```
x += my_val;
```

must be made atomic so that one thread completes the update before another thread performs the conflicting update.

Requirement

Thread 0	Thread 1
---- Update x ----->	
Complete	



The two conflicting updates should not overlap in a way that causes one update to be lost.

The notes identify several ways of ensuring that an operation is atomic; one important method is to ensure that **only one thread executes the update at a time**.

5. Critical Section

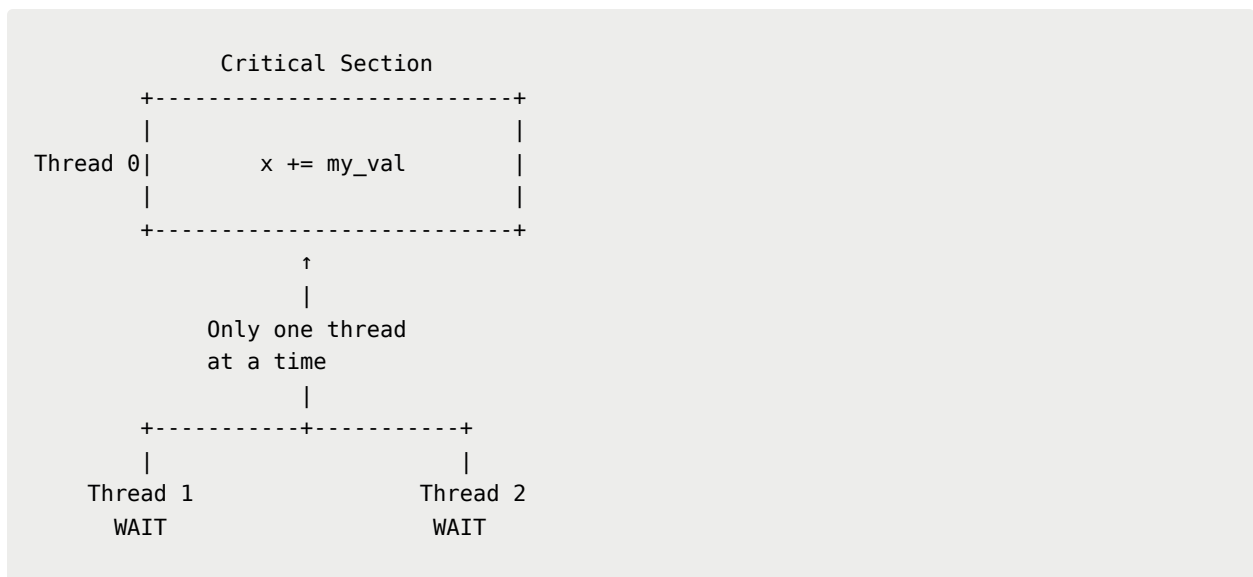
Definition

A **critical section** is a block of code that can be executed by **only one thread at a time**.

The programmer must ensure **mutually exclusive access** to the critical section.

If one thread is executing the critical section, other threads must be excluded from it.

Diagram



Thus, a critical section protects an operation that accesses shared data from simultaneous execution by multiple threads.

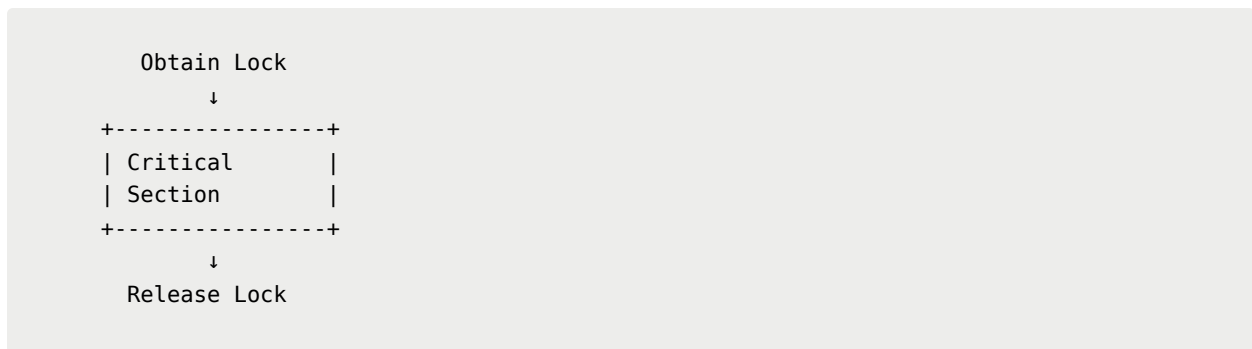
6. Mutex Lock

Definition

The most commonly used mechanism for ensuring **mutual exclusion** is a **mutual exclusion lock**, called a **mutex** or simply a **lock**.

A mutex is a special type of object that has support in the underlying hardware.

The basic idea is:



7. Working of Mutex Lock

Each critical section is protected by a mutex.

Procedure

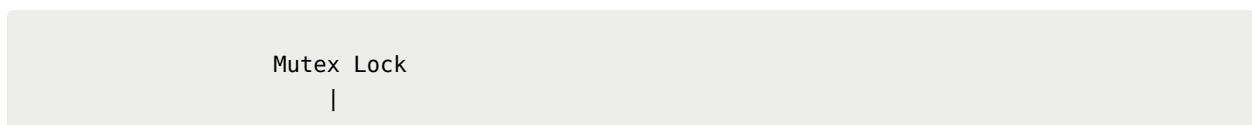
Step 1: Before entering the critical section, a thread must **obtain the mutex** by calling a mutex lock function.

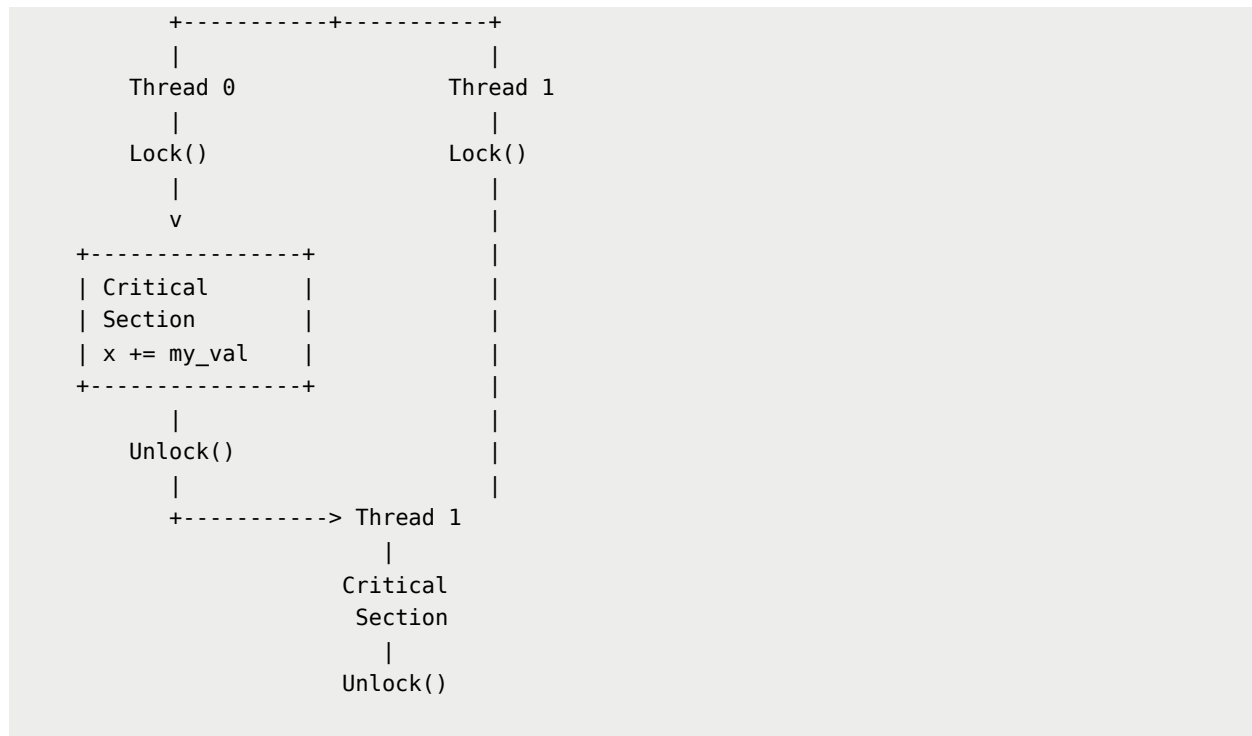
Step 2: The thread executes the code in the critical section.

Step 3: After completing the critical section, the thread **relinquishes the mutex** by calling an unlock function.

Step 4: If another thread attempts to enter the critical section while the mutex is owned by another thread, it waits in its call to the lock function.

Diagram





While Thread 0 owns the lock, Thread 1 must wait before entering the critical section.

8. Example Using Mutex

The shared-variable update are as follows:

```

my_val = Compute_val(my_rank);

Lock(&add_my_val_lock);

x += my_val;

Unlock(&add_my_val_lock);

```

This ensures that **only one thread at a time** can execute:

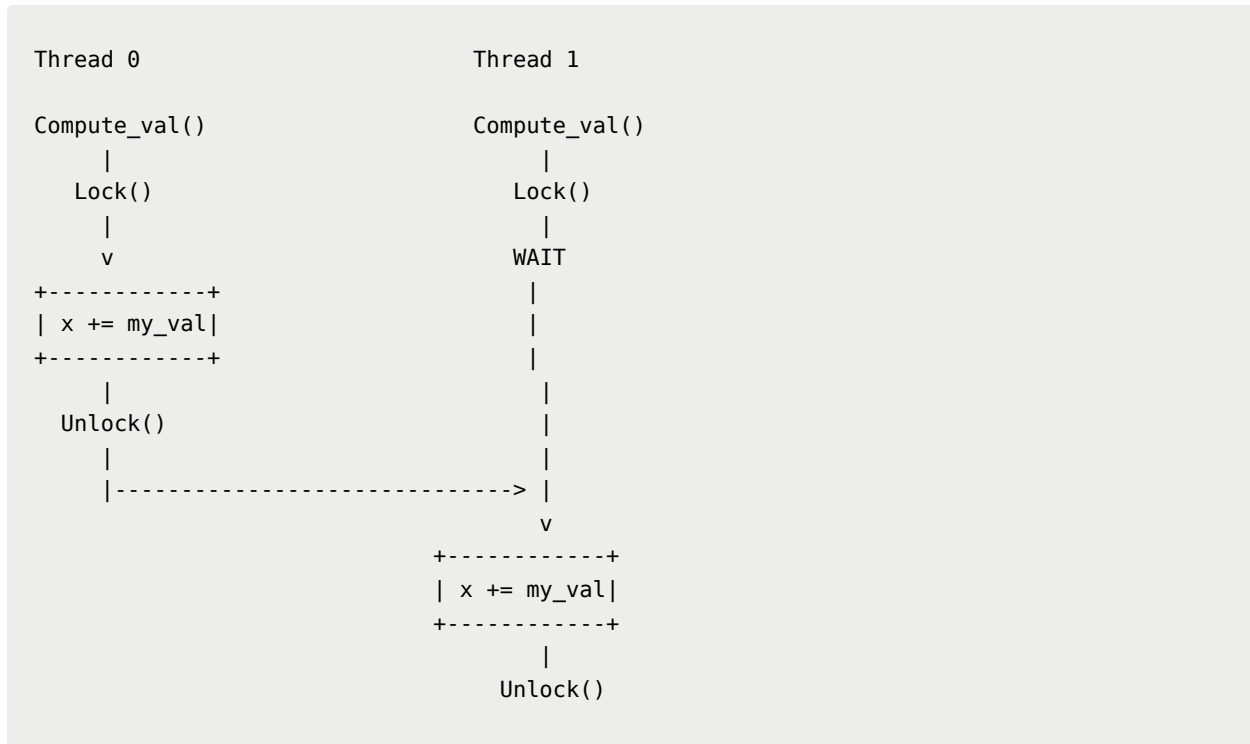
```

x += my_val;

```

Therefore, the update is protected from the race condition.

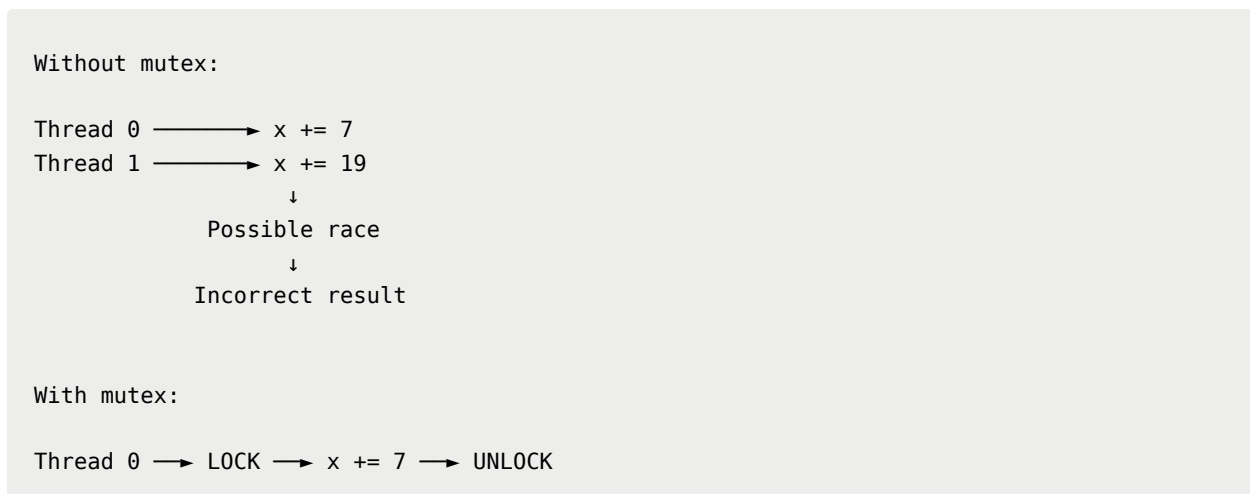
Execution

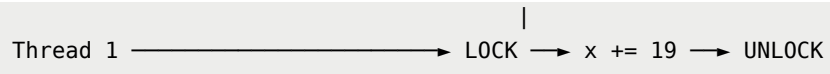


The mutex does **not** impose a predetermined order on the threads. Either Thread 0 or Thread 1 can execute the critical section first.

9. Mutual Exclusion

Mutual exclusion means that only one thread can execute a protected critical section at any given time.





Thus, the mutex ensures that conflicting updates to the shared variable do not execute simultaneously.
